

Metody simplifikace trojúhelníkových sítí

Methods for the simplification of triangular meshes

Oskar Martiňák

Bakalářská práce

Vedoucí práce: Ing. Jan Zapletal, Ph.D.

Ostrava, 2025

Zadání bakalářské práce

Student:

Oskar Martiňák

Studijní program:

B0541A170008 Výpočetní a aplikovaná matematika

Téma:

Metody simplifikace trojúhelníkových sítí
Methods for the simplification of triangular meshes

Jazyk vypracování:

čeština

Zásady pro vypracování:

Simplifikace trojúhelníkových sítí je nedílnou součástí modelování 3D světa. Vstupem do algoritmu je jemná síť, která dobře aproximuje zkoumaný objekt, pro další potřeby (streamování po síti, vizualizace, archivace) je ovšem příliš datově objemná. Výstupem je potom síť, která v jistém smyslu stále dobře modeluje původní objekt, ovšem s menším počtem elementů.

Cílem této práce je seznámit se se základními i moderními přístupy k simplifikaci trojúhelníkových sítí a některé z těchto metod implementovat. Konkrétně by práce měla obsahovat

- 1) přehled základních metod simplifikace, představení konceptu kvadrik,
- 2) implementaci vybraných metod (po konzultaci se školitelem např. Garland-Heckbert, Lindstrom-Turk, spektrální simplifikace),
- 3) porovnání výstupů na vybraných sítích.

Implementace by měla stavět nad standardními reprezentacemi trojúhelníkových sítí, např. half-edge konceptem implementovaným v knihovně OpenMesh. Bonusovým výstupem práce může být příspěvek open-source komunitě ve formě integrace některých metod do knihovny OpenMesh.

Seznam doporučené odborné literatury:

- [1] Michael Garland and Paul S. Heckbert. 1997. Surface simplification using quadric error metrics. In Proceedings of the 24th annual conference on Computer graphics and interactive techniques (SIGGRAPH '97). ACM Press/Addison-Wesley Publishing Co., USA, 209–216. <https://doi.org/10.1145/258734.258849>
- [2] P. Lindstrom and G. Turk, "Fast and memory efficient polygonal simplification," Proceedings Visualization '98 (Cat. No.98CB36276), Research Triangle Park, NC, USA, 1998, pp. 279-286, doi: 10.1109/VISUAL.1998.745314.
- [3] Lescoat, T., Liu, H.-T.D., Thiery, J.-M., Jacobson, A., Boubekur, T. and Ovsjanikov, M. (2020), Spectral Mesh Simplification. Computer Graphics Forum, 39: 315-324. <https://doi.org/10.1111/cgf.13932>

a další literatura po konzultaci se školitelem

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Jan Zapletal, Ph.D.**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2025

Garant studijního programu: prof. RNDr. Jiří Bouchala, Ph.D.

V IS EDISON zadáno: 22.11.2024 14:06:31

Abstrakt

Simplifikační algoritmy 3D sítí jsou dnes nepostradatelné ve světě renderování 3D objektů. Dokážou totiž značně zjedodušit daný objekt – tím mnohonásobně snižují výpočetní nároky na jejich vykreslování a dále také zabírají v paměti počítače daleko méně místa. To pak umožňuje efektivnější práci s těmito objekty, rychlejší běh programů. Tato práce se bude zabývat popisem 3 vybraných algoritmů na simplifikaci trojúhelníkových sítí – Garland-Heckbert algoritmus, bezpaměťová (Memoryless) simplifikace a spektrální simplifikací. Dále dojde na porovnání vybraných algoritmů a popsání jejich výhod a nevýhod při různých konfiguracích daných algoritmů.(1)

Klíčová slova

typografie; L^AT_EX; diplomová práce

Abstract

This is English abstract.

Keywords

simplifikace trojúhelníkové sítě, Garland-Heckbert, Lindstrom-Turk, spektrální simplifikace, simplifikační algoritmy

Obsah

1	Úvod	7
2	Sítě	8
2.1	Definice sítě	8
2.2	Značení	9
2.3	Reprezentace sítě v paměti	9
3	Simplifikační algoritmy	14
3.1	Decimace vrcholů	14
3.2	Shlukování vrcholů	14
3.3	Iterativní kontrakce hran	15
4	Garland-Heckbert simplifikace	17
4.1	Odvození ceny k odstranění hrany	17
4.2	Hledání ideální pozice vrcholu po odstranění hrany	19
4.3	Přepočítávání chyby po odstranění hrany	19
4.4	Kvadriky	20
4.5	Existence ostrého lokálního minima	20
5	Lindstrom-Turk simplifikace	22
5.1	Hledání omezení pro výsledný vrchol	22
5.2	Cena za odstranění hrany	31
5.3	Přepočítávání chyby po odstranění hrany	31
6	Spektrální simplifikace	32
6.1	Diskrétní Laplaceův operátor	32
6.2	Odvození ceny za odstranění hrany	33
6.3	Hledání ideální pozice vrcholu	35
6.4	Restrikční matice	35
6.5	Přepočítání hran a úprava matic	37

7 Pravidelnými ovce dosavadní	38
8 Technické detaily	40
8.1 Křížové odkazy	40
8.2 Jak citovat	40
8.3 Překlad	41
9 Závěr	42
Literatura	43
Přílohy	45

Kapitola 1

Úvod

V současnosti jsme schopni v mnoha technických odvětvích vytvářet 3D modely reálných objektů ve velmi vysoké kvalitě. Pomocí moderních technologií, jakými jsou například počítačová tomografie (CT), magnetická rezonance (MRI), radary satelitů nebo georadary (GPR) takto dokážeme vygenerovat model, který může mít i jednotky miliard (zdroj???) polygonů. Ačkoliv tyto modely velmi zdařile kopírují realitu, jejich extrémní úroveň detailu jde ruku v ruce se zvýšenými nároky na výpočetní výkon a paměť. Ne vždy ale potřebujeme takto vysoké rozlišení. V některých případech by bylo dobré mít o něco méně detailní model, čímž bychom mohli zrychlit jeho zobrazování (rendering) a navíc ulehčit paměti počítače. Přesně toto nám umožňují takzvané simplifikační algoritmy.

Simplifikační neboli zjednodušovací algoritmus načte 3D model objektu a podle určitých metrik se snaží daný model zjednodušit tak, aby se s ním lépe pracovalo, zabíral méně místa v paměti, ale aby přesto co nejvíce připomínal vzhledem originál. Právě kvůli neustálému zvětšování detailu skenovacích zařízení jsou simplifikační algoritmy používány stále častěji. V této bakalářské práci se zaměřím na tři vybrané metody – Garland-Heckbert (GH), Lidstrom-Turk (LT) a spektrální simplifikaci. Cílem metod Garland-Heckbert a Lindstrom-Turk je především zachování vzhledu modelu, avšak spektrální simplifikace se snaží zachovat spíše jeho vnitřní vlastnosti.

3D modely budeme reprezentovat pomocí takzvaných sítí, tedy objekty obsahující vrcholy, hrany a stěny. Budeme uvažovat pouze trojúhelníkové sítě, to znamená sítě pouze s trojúhelníkovými stěnami, jelikož jakákoliv jiná síť mající obecné mnohoúhelníky se dá pomocí dalších metod převést na trojúhelníkovou během předzpracovací fáze.

Součástí práce bude obecná definice sítí a simplifikačních metod, následovat bude přesný popis a odvození vybraných algoritmů a nakonec dojde na jejich porovnání podle několika kritérií. Každou metodu jsem naprogramoval v jazyce C++ nad knihovnou OpenMesh, určenou pro práci se sítěmi.

Kapitola 2

Sítě

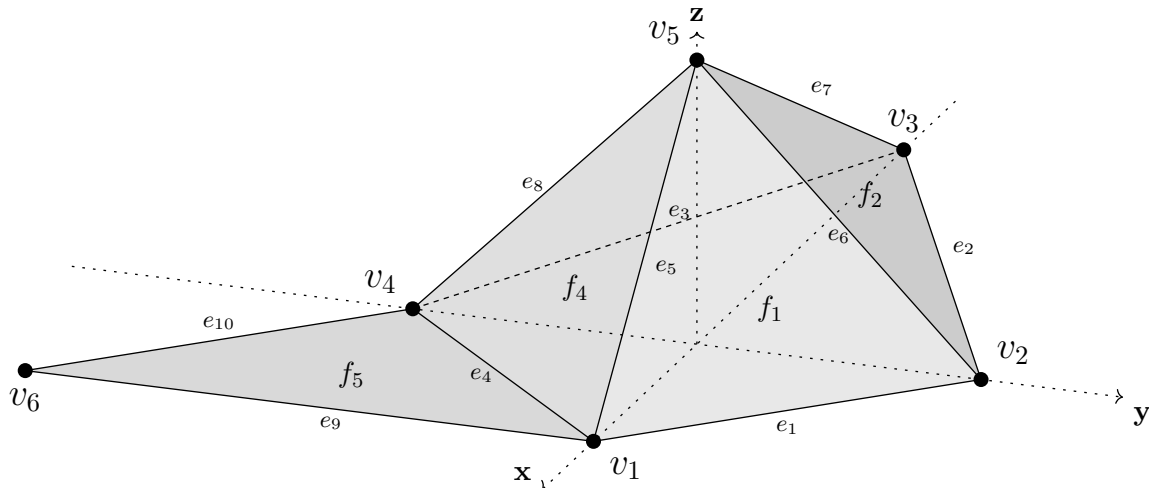
V této kapitole si zavedeme pojem *sítě* z matematického hlediska a uvedeme si příklady možných reprezentací tohoto objektu v paměti počítače.

2.1 Definice sítě

V 3D počítačové grafice definujeme polygonovou síť (angl. *polygon mesh*) jako množinu vrcholů, hran a polygonálních neboli mnohoúhelníkových stěn. Vrchol (*vertex*) je bod v prostoru, hrana (*edge*) je spojení mezi dvěma vrcholy a tvoří tím pádem úsečku. n -úhelníková stěna (*face*) je rovinný útvar o n vrcholech ohraničený n hranami. V tomto textu budeme uvažovat pouze trojúhelníkové sítě, to znamená síť obsahující pouze trojúhelníkové stěny. Snadno si totiž rozmyslíme, že jakýkoliv mnohoúhelník jsme schopni rozdělit na trojúhelníky. Bez újmy na obecnosti proto můžeme s jakoukoliv sítí pracovat jako s trojúhelníkovou – stačí ji upravit pomocí vhodného algoritmu.

Vrcholy, hrany a stěny si taktéž můžeme definovat pomocí d -simplexů, tedy zobecněním trojúhelníku do d dimenzí:

- 0-simplex = **vrchol** – značit jej budeme písmenem v . Souřadnice vrcholu v v prostoru, to znamená vektor o velikosti 3, označíme $\mathbf{v} \in \mathbb{R}^3$.
- 1-simplex = **hrana** – je množina 2 vrcholů sítě, označme $e = \{v_1, v_2\}$. Pro stěny budeme potřebovat orientované hrany, definujme je jako uspořádanou dvojici vrcholů $\vec{e} = (v_1, v_2)$.
- 2-simplex = **stěna** – je klasický trojúhelník. Definujme ji jako množinu 3 jejích orientovaných hran, tedy $f = \{\vec{e}_1, \vec{e}_2, \vec{e}_3\}$, jednodušeji můžeme značit jako uspořádanou trojici vrcholů $f = (v_1, v_2, v_3)$. Orientaci každé stěny budeme potřebovat dále v textu v podobě směru normálového vektoru a taktéž se hodí při renderování sítě, aby bylo jasné, co je vnitřek a co vnějšek sítě. Obecně ale samozřejmě není nutné mít prvky sítě orientované, podobně jako například v grafu.



Obrázek 2.1: Příklad jednoduché 3D trojúhelníkové sítě se 6 vrcholy

2.2 Značení

Pro jednoduché značení různých množin vrcholů, hran a stěn, jež bychom jinak složitě vyjadřovali slovně, si zavedeme simplexové operátory $\lfloor s \rfloor$ a $\lceil s \rceil$. Jestliže máme n -simplex s , pak $\lfloor s \rfloor$ bude značit množinu $n - 1$ -simplexů jež v jistém smyslu tvoří s . Naopak $\lceil s \rceil$ označuje množinu všech jeho $n + 1$ -simplexů. Například pro hranu $e = \{v_1, v_2\}$ platí $\lfloor e \rfloor = \{v_1, v_2\}$ a $\lceil e \rceil = \{t_1, t_2\}$ (kde t_1 a t_2 jsou dvě sousední stěny dané hrany). Intuitivně můžeme tento operátor rozšířit i pro množiny simplexů stejné dimenze:

$$S = \{s_1, s_2, \dots\} \quad \lfloor S \rfloor = \{\lfloor s_i \rfloor : s_i \in S\}$$

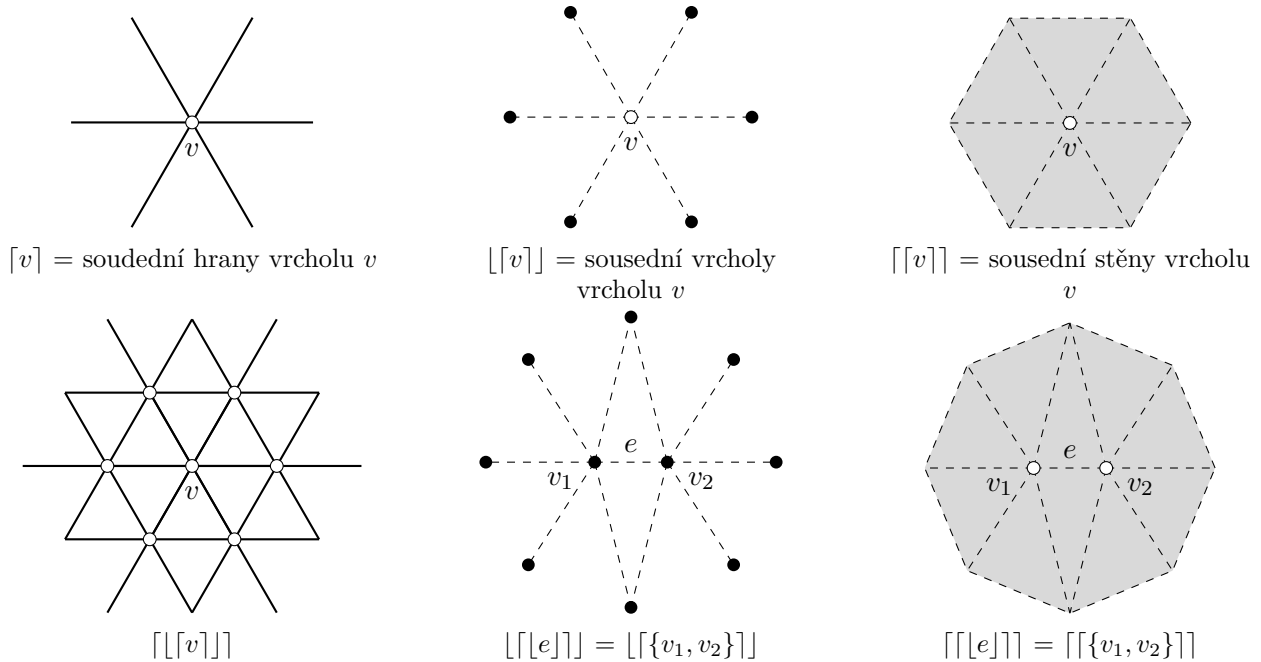
Tímto můžeme operátory skládat, například množina $\lceil \lfloor v \rfloor \rceil = \lceil \{e_1, e_2, \dots, e_k\} \rceil = \{t_1, t_2, \dots, t_l\}$ značí sousední stěny vrcholu v . Nakonec zadefinujeme hranici množiny jako všechny hrany z dané množiny, které mají pouze 1 sousední stěnu, neboli $\partial S = \{e \in S : |\lceil e \rceil| = 1\}$. Hranicí celé sítě pak bude množina všech hran s jednou sousední stěnou. Příklady výše popsaného značení, které budeme používat nejčastěji, shrnuje obrázek 2.2.

2.3 Reprezentace sítě v paměti

Podobně jako u grafů existuje i u sítí několik způsobů reprezentace v paměti počítače.

Vertex-Vertex

Síť je reprezentována množinou vrcholů, z nichž každý obsahuje svou pozici a množinu vrcholů, se kterými je spojen hranou. Jedná se o nejjednodušší způsob reprezentace sítě, jež zabírá málo místa v paměti a dá se jednoduše „tvarovat“. Značnou nevýhodou je však schovaná informace o hranách a



Obrázek 2.2: Příklady množin prvků sítě vytvořených pomocí simplexových operátorů.

Tabulka 2.1: Zápis sítě na obrázku 2.1 pomocí Vertex-Vertex datové struktury

vrchol	souřadnice	sousední vrcholy
v_1	$[1, 0, 0]$	v_1, v_2, v_4, v_5, v_6
v_2	$[0, 1, 0]$	v_1, v_2, v_3, v_5
v_3	$[-2, 0, 0]$	v_2, v_3, v_4, v_5
v_4	$[0, -1, 0]$	v_1, v_3, v_4, v_5, v_6
v_5	$[0, 0, 1]$	v_1, v_2, v_3, v_4, v_5
v_6	$[1, -2, 0]$	v_1, v_4

stěnách, které bychom získali až složitým průchodem celé sítě, což není zrovna praktické. Názorný příklad zápisu sítě na obrázku 2.1 je uveden v tabulce 2.1

Face-Vertex

Jedná se nejpoužívanější způsob vyjádření sítě v počítači. Máme množinu vrcholů s jejich pozicí a sousedními stěnami a taky množinu stěn, které naopak obsahují 3 vrcholy, jimiž jsou tvořeny. Na rozdíl od Vertex-Vertex sítě tedy máme explicitně vyjádřeny navíc i stěny sítě, bohužel hrany musíme zase dohledávat. Konkrétní příklad zápisu je v tabulce 2.2. Podobným způsobem fungují nejběžnější formáty souborů na ukládání sítě. Formáty OBJ a PLY nejprve zapíší souřadnice všech vrcholů sítě a pak z jejich pořadových indexů v zápise sestaví všechny existující stěny.

Tabulka 2.2: Zápis sítě na obrázku 2.1 pomocí Face-Vertex datové struktury

množina vrcholů			množina stěn	
vrchol	souřadnice	sousední stěny	stěna	sousední vrcholy
v_1	$[1, 0, 0]$	f_1, f_4, f_6	f_1	v_1, v_2, v_5
v_2	$[0, 1, 0]$	f_1, f_2	f_2	v_2, v_3, v_5
v_3	$[-2, 0, 0]$	f_2, f_3	f_3	v_3, v_4, v_5
v_4	$[0, -1, 0]$	f_3, f_4, f_5	f_4	v_4, v_1, v_5
v_5	$[0, 0, 1]$	f_1, f_2, f_3, f_4	f_5	v_1, v_4, v_6
v_6	$[1, -2, 0]$	f_5		

Pro představu si v tabulce 2.3 ukažme síť na obrázku 2.1 zapsanou ve formátu OBJ. Poznamenejme, že nejen pro programy na zobrazování sítí je důležité pořadí vrcholů u každé stěny (tzn. její orientace). To totiž určuje směr normálového vektoru, který pak říká, zda se jedná o vnějšek nebo vnitřek sítě. V příkladovém OBJ souboru jsou normálové vektory všech stěn orientovány v kladném směru osy z , proto se v obrázku 2.1 díváme na vnějšek stěny. Pokud bychom ale změnili zápis u f_1 na „f 2 1 5“, normálový vektor by jako jediný směřoval do záporného směru osy z (konkrétně do počátku), což by způsobilo problémy s vykreslováním sítě a prováděním výpočtů na síti. V mnoha případech je tedy nutné dodržet jednotnou orientaci všech stěn.

Hlavní nevýhoda tohoto způsobu reprezentace (stejně jako u VV sítí) se ale ukáže, když budeme chtít nějak modifikovat *konektivitu* sítě, to znamená odstranit nebo přidat vrchol, hranu či stěnu. Jak je už možná zřejmé a jak si taky vysvětlíme podrobněji v další sekci, právě operace odstraňování bude základním kamenem všech simplifikačních algoritmů. Proto by se nám na práci se sítí hodily spíše následující reprezentace.

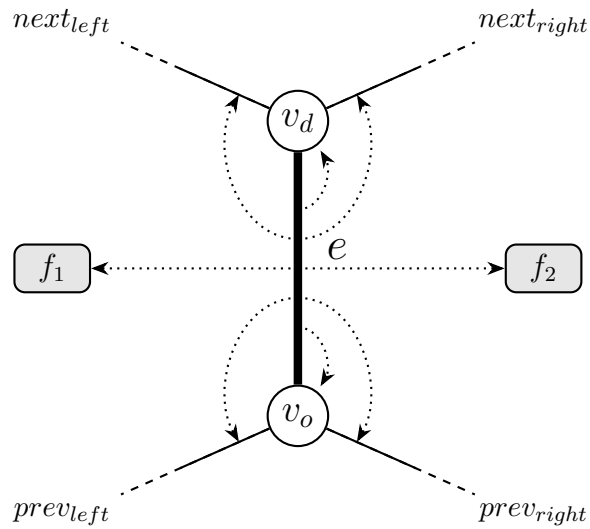
řádek	síť.obj
1	v 1 0 0
2	v 0 1 0
3	v -2 0 0
4	v 0 -1 0
5	v 0 0 1
6	v 1 -2 0
7	
8	f 1 2 5
9	f 2 3 5
10	f 3 4 5
11	f 4 1 5
12	f 1 4 6

Tabulka 2.3: Zápis sítě na obrázku 2.1 ve formátu OBJ.

Winged-edge

Winged-edge reprezentace je už poměrně složitý formát ukládání, který zabírá o dost více místa v paměti než předchozí příklady. Na druhou stranu se ale hodí právě na dynamické změny konektivity sítě, proto jsou často používány v modelovacích programech. Hlavní součástí této reprezentace je množina hran, z nichž každá obsahuje informace o 2 jejích vrcholech, všech stěnách (poznamenejme, že jich může být ≥ 1) a 2 předchozích a 2 následujících hranách, jak se znázorněno na obrázku 2.3.

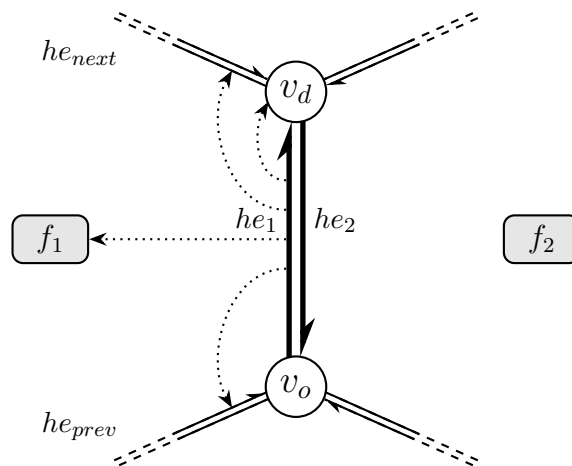
Dále jsou zde množiny vrcholů a stěn, které pro každý prvek obsahují informace o jeho sousedních hranách.



Obrázek 2.3: Datová struktura jedné hrany ve formátu Winged-edge.

Half-edge

Tato struktura sítě je velmi podobná předchozí winged-edge reprezentaci. Jejími základními prvky nejsou hrany, ale half-edges neboli „polohrany“. Ty vzniknou rozseknutím jedné hrany sítě na 2 orientované hrany, každá směřující opačným směrem. Stejně tak se rozdělí i informace, jaké jsou uchovány u každé „polohrany“ – bude obsahovat pouze 1 předchozí a 1 následující hranu, 1 stěnu a 1 vrchol, ke kterému směřuje. Nakonec má taktéž informaci o vedlejší „polohraně“. Přesnou strukturu ilustruje obrázek 2.4.



Obrázek 2.4: Datová struktura polohrany he_1 ve formátu Half-edge.

Hlavní výhodou winged-edge a half-edge reprezentace je právě snadná modifikace konektivity sítě, jež budeme během simplifikace velmi často upravovat. Half-edge strukturu používá knihovna OpenMesh, nad kterou byly naprogramovány simplifikační algoritmy popsané dále v tomto textu.

Kapitola 3

Simplifikační algoritmy

Simplifikační algoritmy používáme ke zjednodušení práce s až moc velkými sítěmi, jejichž zobrazování by trvalo velmi dlouho a zabíraly by až příliš paměti počítače. Vstupem simplifikačního algoritmu bude síť, kterou chceme zjednodušit a výstupem bude síť simplifikovaná. Přirozeně se budeme snažit, ať je výstupní síť co nejpodobnější originálu, jinak řečeno, ať ztratíme co nejméně informací o původní síti.

Podstatou takových algoritmů je zjednodušení neboli simplifikace sítě, což je realizováno odstraněním určitých prvků sítě – vrcholů, hran nebo stěn. Způsobů, kterými můžeme prvky odstraňovat je několik. Zde si popíšeme ty nejběžnější.

3.1 Decimace vrcholů

Decimace vrcholů (*vertex decimation*) si na začátku vezme množinu všech vrcholů sítě. V každém kroku se podle určitého kritéria zvolí 1 vrchol, odstraní se ze sítě a společně s ním se odstraní i jeho sousední stěny. Tímto vznikne v síti díra, která se pak vyplní trojúhelníkovými stěnami. Tento způsob poskytuje dobrý poměr mezi rychlostí a kvalitou výstupní sítě, avšak selhává u sítí, jež nejsou varietami, tzn. u sítí, obsahující hrany, které mají více než 2 sousední stěny ($|[e]| > 2$).

3.2 Shlukování vrcholů

Přístup nazvaný shlukování vrcholů (*vertex clustering*) se jako jeden z mála dá použít i na obecné polygonální sítě. Síť je nejprve pomyslně rozdělena na několik částí krychlovitou mřížkou (*bounding box*) a v každé části pak dojde ke shluku všech vrcholů do jednoho. Nakonec se dotvoří potřebné stěny. S tímto přístupem jsme schopni zjednodušit síť velmi rychle, avšak kvalita je většinou nízká. Navíc nejsme schopni specifikovat přesný počet prvků ve výsledné síti – tuto hodnotu ovlivníme jen nepřímo velikostí mřížky.

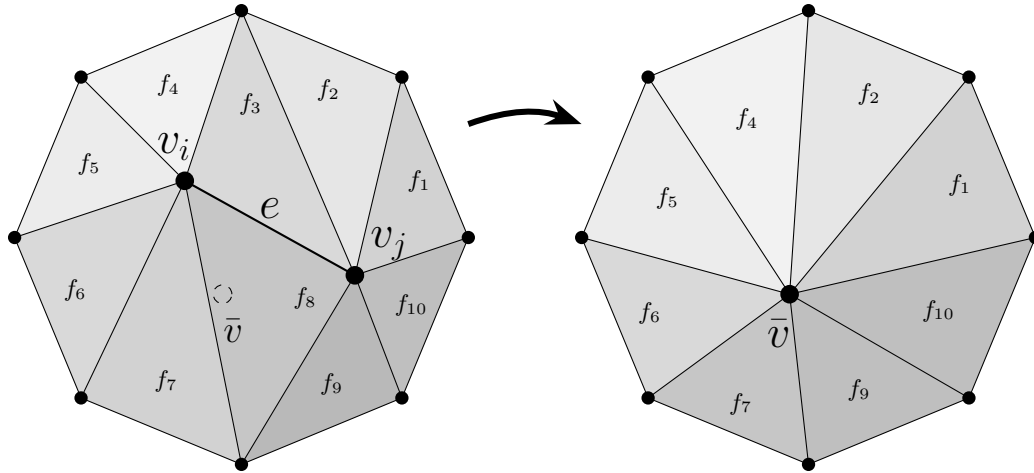
3.3 Iterativní kontrakce hran

Iterativní kontrakce hran (*iterative edge contraction*) je využívána ve všech 3 zde popsaných simplifikačních algoritmech – Garland-Heckbert, Lindstrom-Turk i spektrální simplifikaci. Jejím principem je postupné odstraňování hran a nahrazování je jedním vrcholem. Tento přístup začíná s množinou všech hran sítě $\mathcal{E}$. Podle určitého kritéria se každé hraně $e \in \mathcal{E}$ přiřadí

1. **pozice výsledného vrcholu**, jež nahradí hranu e (označme $\bar{v}$) a
2. **velikost chyby**, kterou bychom způsobili její kontrakcí do vrcholu $\bar{v}$ (označme $E(e, \bar{v})$).

Pojem „chyba“ zde může nést více významů, neboť záleží, podle jakého kritéria jsme chybu stanovili. Například Garland-Heckbert a Lindstrom-Turk vnímají velikost chyby jako nějakou míru, jak moc se změní určité geometrické vlastnosti sítě (objem, vzdálenosti, apod.), avšak třeba spektrální simplifikace za chybu považuje změnu spektrálních vlastností sítě. Přirozeně budeme chtít, ať ve vrcholu $\bar{v}$ je daná chyba co nejmenší.

Dále se všechny hrany seřadí do fronty podle jejich velikosti chyby. V každém kroku pak vezmeme hranu s nejnižší chybou a nahradíme ji jejím vrcholem $\bar{v}$. V každé iteraci tak odstraníme ze sítě 1 vrchol, 2 stěny $[e]$ a 3 hrany. Obrázek 3.1 ilustruje kontrakci hrany $e = \{v_i, v_j\} \rightarrow \bar{v}$.



Obrázek 3.1: Kontrakce hrany $e = \{v_i, v_j\}$ do vrcholu $\bar{v}$.

Po každé kontrakci musíme taktéž přepočítat velikost chyby některým hranám. Jelikož došlo k lokální změně geometrie sítě (konkrétně se změnily stěny $[[\bar{v}]]$), určité hrany mají vypočítanou chybu pro již neexistující strukturu sítě. Podle našich kritérií výpočtu tedy musíme stanovit, jakých hran se to týká, těmto hranám přepočítat chybu a znovu seřadit frontu. Teprve pak můžeme přejít na další odstraňování hrany.

Algoritmus ukončíme, když se nám povedlo odebrat ze sítě určitý počet prvků. Jelikož odstraněním 1 hrany odebereme vždy 1 vrchol, vyplatí se použít počet vrcholů jako naše měřítko.

Případně můžeme jako ukončovací parametr nastavit také maximální hodnotu chyby, kterou nechceme přesáhnout. Celkový postup iterativní kontrakce hran proto můžeme vyjádřit následujícím pseudokódem:

Algorithm 1 Simplifikace sítí pomocí iterativní kontrakce hran

Vstup: síť $\mathcal{M} = (\mathcal{V}, \mathcal{E}, \mathcal{F})$, kde $\mathcal{V}$ jsou vrcholy, $\mathcal{E}$ hrany a $\mathcal{F}$ stěny sítě; cílový počet vrcholů N ; maximální velikost chyby E_{max}

for $e \in \mathcal{E}$ **do**

 vypočítej pozici vrcholu $\bar{v}$ a přiřaď k hraně e

 vypočítej velikost chyby $E(e, \bar{v})$ a přiřaď k hraně e

 přidej hranu e do fronty Q

end for

$\tilde{\mathcal{M}} \leftarrow \mathcal{M}$

seřaď frontu Q dle hodnoty E a vyber hranu e_{min} s nejnižší chybou

while $|\tilde{\mathcal{V}}| > N$ **and** $E(e_{min}, \bar{v}) < E_{max}$ **do**

 proved' kontrakci hrany e_{min} a aktualizuj síť $\tilde{\mathcal{M}}$

 sestav množinu hran $\mathcal{E}_{recalc}$, kterým se musí přepočítat chyba E

for $e \in \mathcal{E}_{recalc}$ **do**

 vypočítej pozici vrcholu $\bar{v}$ a přiřaď k hraně e

 vypočítej velikost chyby $E(e, \bar{v})$ a přiřaď k hraně e

end for

 seřaď frontu Q dle hodnoty E a vyber hranu e_{min} s nejnižší chybou

end while

Výstup: zjednodušená síť $\tilde{\mathcal{M}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}, \tilde{\mathcal{F}})$.

Kapitola 4

Garland-Heckbert simplifikace

Algoritmus Garland-Heckbert je jeden z nejznámějších algoritmů pro simplifikace 3D sítě. Byl odvozen v roce 1997 (dopiš) Garlandem a Heckbertem. Je výpočetně velmi nenáročný a přesto generuje velmi zdařilé zjednodušené modely.

4.1 Odvození ceny k odstranění hrany

Jelikož GH je algoritmem, který používá iterativní kontrakci hran, potřebuje pro svůj chod mít k dispozici velikost chyby, která nastane při odstranění dané hrany, aby mohl vždy vybrat tu s nejnižší chybou a způsobit tak co nejmenší ztrátu informace za cenu zjednodušení sítě.

GH definuje chybu jako součet kvadratické vzdálenosti vrcholu v od jeho sousedních stěn, respektive rovin v prostoru, jež stěny tvoří. Pokud definujeme vrchol jako $\mathbf{v} = [v_x, v_y, v_z, 1]^T$ a stěnu jako $\mathbf{p} = [a, b, c, d]^T$, kde a, b, c a d tvoří koeficienty rovnice dané roviny $p : ax + by + cz + d = 0$ a platí vztah $a^2 + b^2 + c^2 = 1$, dle známého vzorce pro výpočet vzdálenosti bodu a roviny můžeme vzdálenost vyjádřit následovně:

$$d(v, p) = \frac{|av_x + bv_y + cv_z + d|}{a^2 + b^2 + c^2} = \mathbf{v}^T \mathbf{p}$$

Součtem kvadrátu této vzdálenosti pro všechny sousední stěny (označme S) dostaneme velikost

chyby našeho vrcholu:

$$\begin{aligned}
E(\mathbf{v}) &= \sum_{\mathbf{p} \in S} d(\mathbf{v}, \mathbf{p})^2 \\
&= \sum_{\mathbf{p} \in S} (\mathbf{v}^T \mathbf{p})(\mathbf{p}^T \mathbf{v}) \\
&= \sum_{\mathbf{p} \in S} \mathbf{v}^T (\mathbf{p} \mathbf{p}^T) \mathbf{v} \\
&= \mathbf{v}^T \left(\sum_{\mathbf{p} \in S} \mathbf{p} \mathbf{p}^T \right) \mathbf{v} \\
&= \mathbf{v}^T \left(\sum_{\mathbf{p} \in S} K_p \right) \mathbf{v} \\
E(\mathbf{v}) &= \mathbf{v}^T Q \mathbf{v}
\end{aligned} \tag{4.1}$$

kde $K_p = \mathbf{p} \mathbf{p}^T$ je takzvaná fundamentální error kvadrika (viz 4.4), pomocí které se dá vypočítat kvadrát vzdálenosti bodu od roviny. Sečtením pro všechny sousední stěny našeho vrcholu tedy dostaneme kvadriku Q , jenž nám umožňuje počítat součet kvadratických vzdáleností jednoho bodu od několika rovin, přičemž nám stačí uchovávat v paměti pouze jednu 4x4 matici. Tuto matici spočítáme hned na začátku programu pro každý vrchol.

Samozřejmě pokud aplikujeme odvozený vzorec na jakýkoliv vrchol sítě, dostaneme nulový error, neboť daný vrchol je průsečíkem všech jeho sousedních stěn. Nás ale zajímá nahrazování jedné hrany jedním vrcholem, proto je dobré počítat součet kvadrátů vzdáleností od sousedních stěn obou vrcholů hrany. Konkrétně když hranu $(\mathbf{v}_1 \mathbf{v}_2)$ chceme nahradit jedním vrcholem $\bar{\mathbf{v}}$, měli bychom počítat vzdálenosti od sousedních stěn vrcholů $\mathbf{v}_1$ a $\mathbf{v}_2$. Mohli bychom tedy nasbírat všechny takové stěny do jedné množiny a až pak tvořit kvadriku, ale GH volí výpočetně snazší variantu – jednoduše se sečtou kvadriky obou vrcholů. Tedy $\bar{Q} = Q_1 + Q_2$, kde Q_1 a Q_2 jsou kvadriky vrcholů $\mathbf{v}_1$ a $\mathbf{v}_2$ a máme vztah pro výpočet velikosti chyby při odstranění hrany $(\mathbf{v}_1 \mathbf{v}_2)$:

$$E(\bar{\mathbf{v}}) = \bar{\mathbf{v}}^T \bar{Q} \bar{\mathbf{v}} \tag{4.2}$$

Takto nám stačí vypočítat kvadriky už v inicializační části programu a v průběhu odstraňování hran už nic složitějšího nepočítáme – jen se sečtou dvě matice. Tímto samozřejmě GH není tak přesný jak by mohl být, protože některé stěny se budou započítávat do vzdálenosti vícekrát (např. stěny obsahující oba vrcholy $\mathbf{v}_1$ a $\mathbf{v}_2$), za to nám ale ušetří čas a místo v paměti, což bylo v době vzniku tohoto algoritmu žádoucí.

Nyní už jen potřebujeme najít souřadnice vrcholu $\bar{\mathbf{v}}$, abychom mohli error vypočítat a přidat hranu do fronty.

4.2 Hledání ideální pozice vrcholu po odstranění hrany

Ideální pozicí je myšlena pozice, kde je error neboli chyba dané hrany nejmenší. Jednoduše nám tedy stačí najít globální minimum funkce (4.2) na celém definičním oboru. Z podstaty problému víme, že minimum existovat bude (jedná se totiž o součet vzdáleností) a musí být taktéž lokálním minimem, korektní důkaz je uveden v sekci 4.5. K nalezení minima musíme vypočítat parciální derivace dle každé ze tří proměnných a položit je rovny nule. Nejprve si vyjádříme velikost chyby:

$$E(\bar{\mathbf{v}}) = \bar{\mathbf{v}}^T \bar{Q} \bar{\mathbf{v}} = x^2 q_{11} + y^2 q_{22} + z^2 q_{33} + 2xyq_{12} + 2xzq_{13} + 2yzq_{23} + 2xq_{14} + 2yq_{24} + 2zq_{34} + q_{44} \quad (4.3)$$

kde x , y a z jsou souřadnice daného vrcholu a q_{ij} jsou prvky matice $\bar{Q}$, jež je z definice K_p symetrická. Po zderivování, porovnání s nulou a jednoduchých úpravách dostaneme soustavu lineárních rovnic:

$$\begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} \bar{\mathbf{v}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Z naší definice vrcholů, které mají vždy na místě „čtvrté“ souřadnice jedničku, je ke třem rovnicím odvozených z parciálních derivací přidán i čtvrtý řádek. Tuto soustavu následně vyřešíme a máme konečně bod v prostoru, který způsobí nejmenší chybu, pokud se rozhodneme hranu ($\mathbf{v}_1 \mathbf{v}_2$) odstranit. Nyní už můžeme vypočítat i přesnou velikost chyby a hranu pak přidat do fronty.

Mohli bychom taktéž se zaměřit místo celého prostoru pouze na přímku danou hranou ($\mathbf{v}_1 \mathbf{v}_2$) a hledat minimum chyby pouze na této přímce. Eventuálně bychom mohli vybírat pouze z bodů $\mathbf{v}_1$, $\mathbf{v}_2$ a středu hrany. Tyto varianty dokážou zrychlit GH algoritmus za cenu menší přesnosti, neboť nalezená ideální pozice vždy bude mít error větší nebo roven situaci, kdy hledáme jeden bod v celém 3D prostoru. Podrobněji se jim věnuji v kapitole Testy a experimenty???

4.3 Přepočítávání chyby po odstranění hrany

Úplně nakonec musíme přepočítat error u sousedních hran výsledného vrcholu $\bar{\mathbf{v}}$. Obecně bychom měli přepočítat ty hrany ($\mathbf{v}_i \mathbf{v}_j$), u kterých se změnila kvadrika $Q_i + Q_j$. To nastane právě pouze pro sousedy $\bar{\mathbf{v}}$, jelikož jeho změněná kvadrika $\bar{Q}$ ovlivní pouze jeho sousední hrany.

4.4 Kvadriky

Kvadriky neboli kvadratické plochy jsou zobecněním kuželoseček pro vyšší dimenze. V d -dimenzionálním prostoru se souřadnicemi $\mathbf{x}^T = [x_1, x_2, \dots, x_d]$ je můžeme vyjádřit vztahem:

$$\mathbf{x}^T Q \mathbf{x} + P \mathbf{x} + R = 0$$

kde Q je $d \times d$ matice P je řádkový vektor velikosti d a R je skalár. Ve 3D prostoru, ve kterém se sítěmi pracujeme, se pak jedná o elipsoid, paraboloid a hyperboloid, tedy 3D varianty elipsy, paraboly a hyperboly. Když si vezmeme rovnici (4.1), vidíme, že po jednoduchých úpravách nám dává rovnici kvadriky se souřadnicemi $\mathbf{x}^T = [x, y, z]$ a parametry

$$Q = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{12} & q_{22} & q_{23} \\ q_{13} & q_{23} & q_{33} \end{bmatrix} \quad P = [2q_{14}, 2q_{24}, 2q_{34}] \quad R = q_{44}$$

Když tedy do naší rovnice dosadíme obecně souřadnice $\mathbf{v} = [x, y, z, 1]^T$, dostaneme všechny body v prostoru, které mají chybu $E(\mathbf{v})$. Jelikož velikost chyby bude pro většinu hran v síti vždy kladná (jedná se o součet kvadrátů vzdáleností), můžeme z této nerovnosti odvodit zajímavou vlastnost:

$$\bar{\mathbf{v}}^T \bar{Q} \bar{\mathbf{v}} = E(\bar{\mathbf{v}}) > 0$$

z $\bar{\mathbf{v}}^T \bar{Q} \bar{\mathbf{v}} > 0$ ihned vidíme, že matice $\bar{Q}$ bude pozitivně definitní, což znamená, že rovnice (4.2) bude pro konkrétní hodnoty chyby tvořit elipsoid. Výjimkou jsou například hrany, jejichž okolí leží v jedné rovině – velikost chyby pak bude nulová, čímž ztratíme pozitivní definitnost a namísto elipsy může funkce reprezentovat i degenerované kvadratické plochy, například plochy válcové.

Hledáním ideální pozice vrcholu tedy dostaneme takový error, že rovnici pak bude vyhovovat pouze jeden bod v prostoru, tedy námi hledaný $\bar{\mathbf{v}}$, čímž jsme dostali minimální error, jelikož více už elipsoid v reálných číslech zmenšit nemůžeme.

4.5 Existence ostrého lokálního minima

Abychom ověřili, že nalezený bod je skutečně lokálním minimem, musíme podle postačující podmínky existence lokálního minima zkonstruovat Hessovu matici (matici druhých parciálních derivací) a ověřit, zdali je pozitivně definitní. Snadno spočítáme derivace funkce (4.3) a dostaneme matici

$$\begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{12} & q_{22} & q_{23} \\ q_{13} & q_{23} & q_{33} \end{bmatrix}$$

Z minulé sekce víme, že celá matice $\bar{Q}$ je pozitivně definitní. Použijeme Sylvestrovo kritérium, které říká, že matice je pozitivně definitní právě tehdy když má všechny vedoucí hlavní subdeterminanty kladné. Tedy pro $\bar{Q}$ to znamená, že:

$$\left| q_{11} \right| > 0 \quad \left| \begin{array}{cc} q_{11} & q_{12} \\ q_{12} & q_{22} \end{array} \right| > 0 \quad \left| \begin{array}{ccc} q_{11} & q_{12} & q_{13} \\ q_{12} & q_{22} & q_{23} \\ q_{13} & q_{23} & q_{33} \end{array} \right| > 0 \quad \left| \begin{array}{cccc} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ q_{14} & q_{24} & q_{34} & q_{44} \end{array} \right| > 0$$

Z toho jasně vidíme, že Hessova matice musí být pozitivně definitní, jelikož Sylvestrovo kritérium je triviálně splněno i pro tuto menší matici. Máme pozitivní definitnost, bod, který jsme našli, je tedy ostrým lokálním minimem.

Kapitola 5

Lindstrom-Turk simplifikace

Jak již název napovídá, jedná se o simplifikační algoritmus, který si neuchovává žádnou informaci o původním stavu sítě. Na rozdíl tedy od GH, který po celou dobu běhu algoritmu měl k dispozici přesnou informaci o stěnách původní sítě schovanou právě v kvadrikách, memoryless simplifikace (ML/LT simp.) začíná po každém odstranění hrany „na novo“.

Princip tohoto algoritmu spočívá v zachovávání objemu sítě a obsahu plochy u její hranice. To znamená, že objem sítě po simplifikaci bude stejný jako objem původní sítě a kdybychom měli v síti například nějaký otvor, bude zachován i pomyslný obsah otvoru.

5.1 Hledání omezení pro výsledný vrchol

Podobně jako v každém z těchto algoritmů hledáme ideální pozici vrcholu $\bar{\mathbf{v}}$, který může nahradit danou hranu ($\mathbf{v}_1\mathbf{v}_2$). LT pracuje s takzvanými omezeními pro každý takový vrchol. Jedno omezení reprezentuje jednu plochu v prostoru, která určuje, kde se ideálně má vrchol nacházet. Tedy například abychom zachovali objem celé sítě, musíme zachovat objem po každé odstraněné hraně. Takto vypočítáme jedno omezení, jelikož je jedno, kde na omezující ploše bude výsledný vrchol ležet – stále budeme mít stejný objem (více detailů v následujících sekcích). Když pak postupně vypočítáme 3 omezení neboli 3 plochy, dostaneme ideální pozici výsledného vrcholu $\bar{\mathbf{v}}$ v průsečíku těchto ploch.

Obecně si i -té omezení označíme vztahem

$$\mathbf{a}_i^T \bar{\mathbf{v}} = b_i$$

kde tentokrát $\bar{\mathbf{v}} = [x, y, z]$, $\mathbf{a}_i$ je vektor o velikosti 3 a b_i je skalár. Tento zápis značí jednu plochu $p_i : a_{i1}x + a_{i2}y + a_{i3}z = b_i$, na které musí výsledný vrchol ležet. Omezení $(\mathbf{a}_i, b_i)$ budeme postupně, za jistých okolností přidávat do systému $(A, \mathbf{b})$, kde jednotlivé $\mathbf{a}_i$ jsou řádky matice A a vektor

$\mathbf{b}$ obsahuje skaláry b_i . Jakmile získáme 3 omezení, můžeme už jednoduše spočítat ideální pozici vrcholu $\bar{\mathbf{v}}$ přes soustavu tří lineárních rovnic.

Jenže z důvodu numerické nepřesnosti, která vzniká například zaokrouhlovací chybou nebo nedostatečně přesnou reprezentací desetinných čísel v počítači, můžou rovnoběžné a „skoro-rovnoběžné“ plochy (tzn. ta omezení) značně zahýbat s přesný výsledkem. LT simplifikace je proto opatřena podmínkami pro přidávání dalších omezení do našeho systému, které by měly nepřesnostem zamezit. Z toho důvodu LT popisuje, jak získat více než tři omezení v případě, že by nějaké omezení neprošlo přes naše podmínky.

Po úplně prvním omezení je vyžadováno, aby nebylo nulové, tedy $\mathbf{a}_1 \neq \mathbf{0}$. Když už v systému máme první omezení a chceme přidávat druhé, je třeba kontrolovat rovnoběžnost rovin. Vezmeme si tedy známý vzorec ze střední školy a vypočítáme, zda normálové vektory již přidané plochy a plochy, kterou právě přidáváme, svírají úhel větší než nějaké malé, námi zvolené α^1 :

$$(\mathbf{a}_1^T \mathbf{a}_2)^2 < (\|\mathbf{a}_1\| \|\mathbf{a}_2\| \cos \alpha)^2$$

Pokud ano, přidáme druhé omezení, pokud ne, víme, že plochy by byly rovnoběžné nebo „skoro-rovnoběžné“ a celkový výsledek by mohl být dosti nepřesný. Druhé omezení tedy zahodíme a pokoušíme se najít vhodnější. Jestli se nám to podaří, zbývá nám přidat poslední omezení. Podmínka pro přidání třetího omezení je velmi podobná – zase kontrolujeme rovnoběžnost rovin:

$$((\mathbf{a}_1 \times \mathbf{a}_2)^T \mathbf{a}_3)^2 > (\|\mathbf{a}_1 \times \mathbf{a}_2\| \|\mathbf{a}_3\| \sin \alpha)^2$$

Tentokrát si pomůžeme vektorovým součinem a budeme chtít, ať normálový vektor třetí plochy je co nejméně kolmý k $\mathbf{a}_1 \times \mathbf{a}_2$, tedy ať je třetí plocha co nejméně rovnoběžná s dvěma již do systému přidanými.

Pokud přes naše podmínky dokážeme naplnit matici A a vektor $\mathbf{b}$ máme vyhráno. Pokud ne, nebudeme přidávat danou hranu do fronty, protože se jeví jako nevhodný kandidát na odstranění, který by potenciálně způsobil větší chybu.

Následující části detailně popisují získávání omezení z různých geometrických vlastností sítě.

5.1.1 Zachování objemu

I když obecně síť nemusí být uzavřená a tudíž nedá se bavit o jejích objemu, můžeme na objem sítě nahlížet jako na prostor, který síť obklopuje. Jeho zachováním pak udržíme stálý 3D tvar a stálou 2D projekci sítě.

Jak již bylo nastíněno výše, abychom zachovali objem celé sítě, musíme jej zachovat při každém odstraňování hrany. To znamená, že lokální objem v prostoru sousedních stěn dvou vrcholů momentálně počítané hrany, musí být stejný jako lokální objem po nahrazení hrany výsledným vrcho-

¹LT simplifikace volí $\alpha = 1^\circ$. Tato hodnota je použita i v mém testování LT simplifikace

lem $\bar{\mathbf{v}}$. Když si představíme, co se přesně děje při kontrakci hrany, můžeme si vzít každou stěnu $t_i = (\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_3^{t_i})$ v našem prostoru jako podstavu čtyřstěnu a $\bar{\mathbf{v}}$ jako jeho vrchol. Dostaneme tudíž čtyřstěn $C_i = (\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_3^{t_i}, \bar{\mathbf{v}})$. Takto vzniklý čtyřstěn svým objemem reprezentuje změnu objemu sítě. Zřejmě pokud vrchol $\bar{\mathbf{v}}$ bude vně sítě, znamená to, že se objem sítě pro danou stěnu zvětší, tedy řekněme, že objem čtyřstěnu bude kladný. Naopak, pokud se $\bar{\mathbf{v}}$ bude nacházet uvnitř sítě, její objem se zmenší a čtyřstěn bude mít záporný objem.

Nyní se konečně dostáváme k zachování objemu – potřebujeme aby součet objemů čtyřstěnů byl nulový, tím pádem bude nulová změna celkového objemu sítě a máme její objem zachován:

$$\sum_i V(C_i) = 0$$

Objem čtyřstěnu získáme ze známého vzorce z analytické geometrie obsahující smíšený součin $V(C_i) = \frac{1}{6} |(\mathbf{u}_1 \times \mathbf{u}_2)^T \cdot \mathbf{u}_3|$, kde vektory $\mathbf{u}$ můžou vycházet například z vrcholu $\bar{\mathbf{v}}$ do vrcholů původní stěny $t_i = (\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_3^{t_i})$, tzn:

$$\mathbf{u}_1 = \mathbf{v}_1^{t_i} - \bar{\mathbf{v}} \quad \mathbf{u}_2 = \mathbf{v}_2^{t_i} - \bar{\mathbf{v}} \quad \mathbf{u}_3 = \mathbf{v}_3^{t_i} - \bar{\mathbf{v}}$$

Navíc potřebujeme rozlišit mezi kladným a záporným objemem, tudíž odstraníme absolutní hodnotu. Po dosazení do sumy dostáváme

$$\sum_i \left(\frac{1}{6} [(\mathbf{v}_1^{t_i} - \bar{\mathbf{v}}) \times (\mathbf{v}_2^{t_i} - \bar{\mathbf{v}})]^T \cdot (\mathbf{v}_3^{t_i} - \bar{\mathbf{v}}) \right) = 0$$

Nyní potřebujeme spočítat neznámou $\bar{\mathbf{v}}$. Po delších úpravách zredukujeme složitý výraz na jednoduše spočítatelné hodnoty:

$$\left(\sum_i [(\mathbf{v}_2^{t_i} - \mathbf{v}_1^{t_i}) \times (\mathbf{v}_3^{t_i} - \mathbf{v}_1^{t_i})]^T \right) \cdot \bar{\mathbf{v}} = \sum_i \begin{vmatrix} v_{1x}^{t_i} & v_{2x}^{t_i} & v_{3x}^{t_i} \\ v_{1y}^{t_i} & v_{2y}^{t_i} & v_{3y}^{t_i} \\ v_{1z}^{t_i} & v_{2z}^{t_i} & v_{3z}^{t_i} \end{vmatrix} \quad (5.1)$$

kde na pravé straně rovnice máme determinant třech vrcholů stěny a $\mathbf{n}_i = (\mathbf{v}_2^{t_i} - \mathbf{v}_1^{t_i}) \times (\mathbf{v}_3^{t_i} - \mathbf{v}_1^{t_i})$ je její normálový vektor. Všimněme si, že rovnice je vyjádřená ve tvaru omezení $\mathbf{a}_i^T \bar{\mathbf{v}} = b_i$. Součtem normálových vektorů přes všechny stěny v prostoru označ??? dostaneme první omezení $\mathbf{a}_1$ a součtem determinantů získáme první skalár b_1 . Ty následně musí projít přes naše podmínky a až pak je přidáme do systému omezení $(\mathbf{A}, \mathbf{b})$.

5.1.2 Zachování obsahu hranice

LT simplifikace si také dává za cíl zachovat obsah hranice. Tím je myšlen obsah útvaru, který daná hranice sítě obklopuje. Pokud například máme rovinnou síť, která tedy reprezentuje nějaký

2D útvar, LT algoritmus zachová obsah tohoto útvaru. Samozřejmě že konkrétní hranice se může měnit, ale výsledný obsah útvaru musí stále zůstat stejný. Podobně když si vezmeme rovinnou síť, jež má uprostřed řekněme díru ve tvaru šestiúhelníku, LT simplifikace zachová obsah tohoto útvaru. Není ale řečeno, že se šestiúhelník nestane někde v průběhu algoritmu pětiúhelníkem nebo čtvercem. I v takovém případě ale bude útvar mít stále stejný obsah.

Omezení pro zachování hranice můžeme počítat pouze tehdy, pokud ??? značení hranice hrany, které právě počítáme její cenu k odstranění je neprázdná. Rovnici pro zachování hranice získáme velmi podobně jako u objemu. Na rovinné síti můžeme každé hraně e_i na hranici přiřadit výsledný vrchol $\bar{\mathbf{v}}$ a takto dostaneme trojúhelník $T_i = (\mathbf{v}_1^{e_i}, \mathbf{v}_2^{e_i}, \bar{\mathbf{v}})$, který bude mít zase buď kladný nebo záporný obsah podle toho, zda se $\bar{\mathbf{v}}$ bude nacházet vně sítě (za hranicí) nebo uvnitř. A stejně jako u zachování objemu chceme, aby součet obsahů všech trojúhelníků (tzn. pro každou hranu z hranice sítě označ???) byl nulový:

$$\sum_i S(T_i) = 0$$

Většinou ale sítě nejsou rovinné, takže odvozená rovnice nedává moc smysl u obecné sítě. Můžeme proto místo toho minimalizovat velikost součtu směrových vektorů každého trojúhelníku. Obsah tedy umocníme a minimalizujeme:

$$\sum_i S(T_i)^2 = \sum_i \left\| \frac{1}{2}(\mathbf{v}_1^{e_i} - \bar{\mathbf{v}}) \times (\mathbf{v}_2^{e_i} - \bar{\mathbf{v}}) \right\|^2 = \frac{1}{4} \left\| (\bar{\mathbf{v}} \times \sum_i (\mathbf{v}_2^{e_i} - \mathbf{v}_1^{e_i}) + \sum_i (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})) \right\|^2$$

Označme si $\mathbf{e}_1 = \sum_i (\mathbf{v}_2^{e_i} - \mathbf{v}_1^{e_i})$, $\mathbf{e}_2 = \sum_i (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})$ a $\mathbf{e}_3 = \mathbf{e}_1 \times \mathbf{e}_2$. Po minimalizaci dostaneme dvě omezení ve tvaru:

$$\begin{aligned} (\mathbf{e}_1^T \mathbf{e}_1 \mathbf{e}_3^T) \bar{\mathbf{v}} &= -\mathbf{e}_3^T \mathbf{e}_3 \\ (\mathbf{e}_1 \times \mathbf{e}_3)^T \bar{\mathbf{v}} &= 0 \end{aligned}$$

Tato omezení pak v libovolném pořadí přidáme do našeho systému, pakliže projdou našimi podmínkami. Nyní už je možné, že jsme pro nějakou hranu nasbírali tři omezení, v takovém případě se tedy můžeme vrhnout na výpočet velikosti chyby pro danou hranu.

5.1.3 Optimalizace objemu

Jestliže v této fázi algoritmu nemáme pro hranu $\mathbf{e}$ nalezeny 3 omezení (což je případ minimálně všech hran, které nemají poblíž hranici sítě a nemohly tak získat 2 omezení ze zachování hrany), musíme definovat další možnosti, ze kterých můžeme zbylá omezení vyvodit.

Objem sice máme zachován, ale bylo by rozumné, kdyby celková změna objemů v absolutní hodnotě byla co nejmenší. Například kdybychom měli rovinnou síť a v jednom místě by byl důlek a hned vedle něj stejně velká vypouklina, zřejmě se dá odstranit hrana tak, aby se celkový objem

zachoval – odstraníme důlek i vypouklinu navzájem. Jenže pochopitelně by bylo vhodnější, kdybychom o tuto informaci nepřišli a místo toho zvolili jinou pozici výsledného vrcholu nebo vybrali podle velikosti chyby úplně jinou hranu.

Už tedy nebudeme rozlišovat objemy čtyřstěnů na kladné a záporné, ale umocníme je na druhou a budeme se snažit minimalizovat jejich absolutní změnu. Definujme proto funkci, která bude hrát roli i v samotném výpočtu velikosti chyby:

$$f_V(\mathbf{e}, \bar{\mathbf{v}}) = \sum_i V(C_i)^2$$

Podobnou úpravou jako v části 5.1.1 a uspořádáním (lze hned vidět z upravené rovnice 5.1, ke které akorát přidáme $\frac{1}{6}$) dostaneme rovnost

$$f_V(\mathbf{e}, \bar{\mathbf{v}}) = \frac{1}{18} \left[\frac{1}{2} \bar{\mathbf{v}}^T \sum_i \mathbf{n}_i \mathbf{n}_i^T \bar{\mathbf{v}} - \sum_i \det(\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_2^{t_i}) \mathbf{n}_i^T \bar{\mathbf{v}} + \frac{1}{2} \sum_i \det(\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_2^{t_i})^2 \right] \quad (5.2)$$

kde

$$\det(\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_2^{t_i}) = \begin{vmatrix} v_{1x}^{t_i} & v_{2x}^{t_i} & v_{3x}^{t_i} \\ v_{1y}^{t_i} & v_{2y}^{t_i} & v_{3y}^{t_i} \\ v_{1z}^{t_i} & v_{2z}^{t_i} & v_{3z}^{t_i} \end{vmatrix}$$

Jde vidět, že daná funkce má zase tvar nějaké kvadriky. Zapišme si ji ve tvaru

$$f_V(\mathbf{e}, \bar{\mathbf{v}}) = \frac{1}{2} \bar{\mathbf{v}}^T H \bar{\mathbf{v}} + \mathbf{c}^T \bar{\mathbf{v}} + \frac{1}{2} k \quad (5.3)$$

kde

$$H = \frac{1}{18} \sum_i \mathbf{n}_i \mathbf{n}_i^T \quad \mathbf{c} = \frac{1}{18} \sum_i \det(\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_2^{t_i}) \mathbf{n}_i^T \quad k = \frac{1}{18} \sum_i \det(\mathbf{v}_1^{t_i}, \mathbf{v}_2^{t_i}, \mathbf{v}_2^{t_i})^2$$

Nyní stejně jako u GH můžeme spočítat parciální derivace, položit je rovny nule a tím získáme tři omezení ve tvaru $H\bar{\mathbf{v}} = -\mathbf{c}$. Všimněme si, že matice H je ve skutečnosti Hessian funkce $f(\mathbf{e}, \bar{\mathbf{v}})$ a $H\bar{\mathbf{v}} + \mathbf{c}$ je jejím gradientem.

Jenže my s nějakou pravděpodobností už několik omezení máme z předešlých částí (jestli jsme se dostali do tohoto bodu, máme 0 až 2 omezení). Proto nemůžeme jenom tak vzít všechna nově vypočítaná omezení a nahradit jimi ty staré. Musíme ta zbylá omezení získat ze vztahu $H\bar{\mathbf{v}} = -\mathbf{c}$ a podmínky $A\bar{\mathbf{v}} = \mathbf{b}$. Tomuto problému se říká úloha kvadratického programování.

5.1.4 Úloha kvadratického programování

V úloze kvadratického programování (angl. *quadratic programming problem*). máme obecně kvadratickou funkci více proměnných kterou musíme optimalizovat, to znamená najít minimum či

maximum, za určitých lineárních omezení. Konkrétně optimalizujeme funkci

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$$

za m podmínek $A\mathbf{x} \preceq \mathbf{b}$, kde Q je $n \times n$ symetrická matice, c je vektor velikosti n , A je matice velikosti $m \times n$ a b je vektor velikosti m . Symbol $\preceq$ označuje nerovnost po složkách, tedy podmínky $A\mathbf{x} \preceq \mathbf{b}$ je jen kompaktně zapsaných m lineárních nerovností. Samozřejmě můžeme mít i podmínky ve tvaru rovnosti $A\mathbf{x} = \mathbf{b}$, úplně stejně jako u našeho problému.

Mezi hlavní metody na řešení úlohy kvadratického programování patří metoda vnitřního bodu, metoda aktivních množin, metoda sdružených gradientů nebo simplexový algoritmus. V našem případě, kdy máme podmínky na rovnost, můžeme použít klasický postup při hledání vázaných lokálních extrému funkce více proměnných – metodu Langrangeových multiplikátorů.

Dle postačující podmínky existence vázaného lokálního extrému musíme nejprve najít stacionární body funkce

$$F(\mathbf{x}) = f(\mathbf{x}) + \sum_i \lambda_i g_i(\mathbf{x})$$

proměnných $\mathbf{x} = [x_1, x_2, \dots, x_d]$, kde $f(\mathbf{x})$ je obecně funkce, jejíž extrémy chceme najít, $g_i(\mathbf{x})$ je i -tá podmínka neboli omezení a $\lambda_i \in \mathbb{R}$ pro $i \in \{1, \dots, n\}$ kde n je počet zadaných podmínek. Označme si množinu M jako průnik všech podmínek:

$$M = \bigcap_i \left\{ \mathbf{x} \in \mathbb{R}^d : g_i(\mathbf{x}) = 0 \right\}$$

Jestliže najdeme nějaký stacionární bod $\mathbf{p} \in M$ a Hessova matice funkce $F(\mathbf{x})$ pro příslušné λ bude v bodě $\mathbf{p}$ pozitivně definitní, víme, že bod $\mathbf{p}$ je ostrým lokálním minimem funkce $f(\mathbf{x})$ vzhledem k množině M .

V našem případě máme funkci následující:

$$F(\bar{\mathbf{v}}) = f_V(\mathbf{e}, \bar{\mathbf{v}}) + \sum_i \lambda_i g_i(\bar{\mathbf{v}})$$

kde $g_i(\bar{\mathbf{v}})$ bude označovat i -té omezení v našem systému $(A, \mathbf{b})$, tedy $g_i(\bar{\mathbf{v}}) = \mathbf{a}_i \bar{\mathbf{v}} - b_i$. Předpokládejme, že máme momentálně n omezení. Tuto funkci rozepíšeme maticově a taktéž spočítáme parciální derivace podle každé ze tří souřadnic, porovnáme s nulou k nalezení stacionárního bodu a taktéž zapíšeme do matic:

$$\begin{aligned} F(\bar{\mathbf{v}}) &= \frac{1}{2}\bar{\mathbf{v}}^T H \bar{\mathbf{v}} + \mathbf{c}^T \bar{\mathbf{v}} + \frac{1}{2}k + \lambda^T (A\bar{\mathbf{v}} - \mathbf{b}) \\ F'(\bar{\mathbf{v}}) &= H\bar{\mathbf{v}} + \mathbf{c} + A^T \lambda = 0 \end{aligned} \tag{5.4}$$

kde λ je vektor λ pro každé naše omezení, tedy je to vektor velikosti n . Nyní bychom rádi z této rovnice a z našich omezení vyjádřili konkrétní λ a konkrétní souřadnice, abychom

měli stacionární bod funkce. Můžeme si pomoci inverzní maticí, kterou bychom potenciálně mohli vynulovat lambda. Pokud do sloupců nové matice Z velikosti 3×3 (připomeňme, že potřebujeme právě 3 omezení k nalezení ideální pozice vrcholu) doplníme řádky matice A a zbylé sloupce vyplníme tak, aby byly ortogonální ke stávajícím sloupcům Z , dostaneme matici, která nebude singulární. Tím pádem dokážeme najít inverzní matici Z^{-1}

Uvědomme si, co přesně dělá inverzní matice. Rozepišme si známý vztah $ZZ^{-1} = Z^{-1}Z = I$ dle naší definice Z například pro $n = 2$:

$$Z^{-1}Z = \begin{bmatrix} z_{11} & z_{12} & z_{13} \\ z_{21} & z_{22} & z_{23} \\ z_{31} & z_{32} & z_{33} \end{bmatrix} \begin{bmatrix} a_{1x} & a_{2x} & z_{3x} \\ a_{1y} & a_{2y} & z_{3y} \\ a_{1z} & a_{2z} & z_{3z} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I$$

Vidíme, že první dva sloupce matice Z jsou řádky A . Když si promyslíme násobení matic, zjistíme, že vynásobením třetího řádku Z^{-1} s prvními dvěma sloupci Z dostaneme pokaždé nulu. To ale znamená, že když ze Z^{-1} získáme pouze třetí řádek a pak jím přenásobíme celou rovnici (5.4), vynuluje se nám lambda a už jenom snadno ze soustavy lineárních rovnic vyjádříme zbylá omezení.

Pouze třetí řádek Z^{-1} získáme přenásobením třetím řádkem jednotkové matice:

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} z_{11} & z_{12} & z_{13} \\ z_{21} & z_{22} & z_{23} \\ z_{31} & z_{32} & z_{33} \end{bmatrix} = \begin{bmatrix} z_{31} & z_{32} & z_{33} \end{bmatrix}$$

Obecně pro n podmínek vezmeme matici $I_{(3-n) \times 3}$ o velikosti $(3-n) \times 3$, kterou vytvoříme odstraněním n horních řádků z jednotkové matice. Rovnici (5.4) tedy vynásobíme $I_{(3-n) \times 3}Z^{-1}$ a dostaneme

$$\begin{aligned} I_{(3-n) \times 3}Z^{-1}(H\bar{\mathbf{v}} + \mathbf{c}) &= 0 \\ (I_{(3-n) \times 3}Z^{-1}H)\bar{\mathbf{v}} &= -(I_{(3-n) \times 3}Z^{-1}\mathbf{c}) \end{aligned} \quad (5.5)$$

z čehož už snadno získáme zbylá omezení do systému $(A, \mathbf{b})$. Konkrétně jich bude $3-n$ a samozřejmě musí zase projít našimi podmínkami.

Rozeberme si ještě detailně případy, které mohou nastat pro různá n .

- $n = 0$: Nemáme zatím žádná omezení, proto nemusíme řešit hledání extrémů vzhledem k nějaké množině M , ale stačí klasicky hledat extrémy funkce $f_V(\mathbf{e}, \bar{\mathbf{v}})$ podobně jako to dělá GH. Dostaneme tři omezení ve tvaru $H\bar{\mathbf{v}} = -\mathbf{c}$.
- $n = 1$: Máme právě jedno omezení, dosadíme jej tedy do prvního sloupce matice Z . Zbylé dva sloupce musí být ortogonální, abychom měli jistotu, že Z bude regulární. Druhý sloupec stačí udělat kolmý k prvnímu, tudíž chceme aby se skalární součin rovnal nule. Jednoduše

stačí zvolit třeba $z_2 = [a_{1y}, -a_{1x}, 0]$. Třetí sloupec získáme vektorovým součinem $z_3 = a_1 \times z_2$, jelikož ten nám dá vektor kolmý k oběma vektorům a_1 a z_2 .

- $n = 2$: Už dvě omezení máme, proto velmi podobně využijeme vektorový součin na třetí sloupec $z_3 = a_1 \times a_2$.

Fakt, že stacionární bod funkce $F(\bar{\mathbf{v}})$, který získáme z omezení $A\mathbf{x} = \mathbf{b}$ a rovnice (5.5), je i minimem funkce $f_V(\mathbf{e}, \bar{\mathbf{v}})$ závisí zase na pozitivní definitnosti Hessovy matice. Po krátkém výpočtu zjistíme, že se rovná právě matici H . Velmi podobným důkazem jako u GH zjistíme, že i H je pozitivně definitní.

5.1.5 Optimalizace hranice

V případě, že některá omezení neprojdou našimi podmínkami, víme, že okolní stěny (označ ???) hrany $\mathbf{e}$ leží v jedné rovině, nebo jsou tomu velmi blízko – jak moc můžou být v rovině určuje velikost parametru α . Z toho důvodu máme poměrně velký stupeň volnosti, co se týče optimalizace objemu, protože na rovinné ploše se objem měnit nebude. Můžeme proto zavést další omezení vyplývající ze sekce 5.1.2. Podobně jako u optimalizace objemu se budeme snažit minimalizovat součet kvadrátů obsahu trojúhelníků, jež značí změnu v obsahu hranice. Tudíž následující postup můžeme využít pouze pokud je v okolí hrany $\mathbf{e}$ (??? značení) hranice sítě. Minimalizujeme funkci

$$f_B(\mathbf{e}, \bar{\mathbf{v}}) = \sum_i S(T_i)^2 = \sum_i \left\| \frac{1}{2}(\mathbf{v}_1^{e_i} - \bar{\mathbf{v}}) \times (\mathbf{v}_2^{e_i} - \bar{\mathbf{v}}) \right\|^2$$

Po umocnění a delších úpravách dostaneme například tvar

$$f_B(\mathbf{e}, \bar{\mathbf{v}}) = \frac{1}{4} \bar{\mathbf{v}}^T \sum_i H_i \bar{\mathbf{v}} + \frac{1}{2} \sum_i \left[(\mathbf{v}_2^{e_i} - \mathbf{v}_1^{e_i}) \times (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})^T \right] \bar{\mathbf{v}} + \frac{1}{4} \sum_i \left[(\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})^T (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i}) \right] \quad (5.6)$$

kde

$$H_i = \begin{bmatrix} 0 & v_{2z}^{e_i} - v_{1z}^{e_i} & v_{1y}^{e_i} - v_{2y}^{e_i} \\ v_{1z}^{e_i} - v_{2z}^{e_i} & 0 & v_{2x}^{e_i} - v_{1x}^{e_i} \\ v_{2y}^{e_i} - v_{1y}^{e_i} & v_{1x}^{e_i} - v_{2x}^{e_i} & 0 \end{bmatrix} \begin{bmatrix} 0 & v_{1z}^{e_i} - v_{2z}^{e_i} & v_{2y}^{e_i} - v_{1y}^{e_i} \\ v_{2z}^{e_i} - v_{1z}^{e_i} & 0 & v_{1x}^{e_i} - v_{2x}^{e_i} \\ v_{1y}^{e_i} - v_{2y}^{e_i} & v_{2x}^{e_i} - v_{1x}^{e_i} & 0 \end{bmatrix}$$

Všimněme si, že je podobná funkci (5.3) ze sekce Optimalizace objemu. Napíšeme si ji proto zase v tomto tvaru a použijeme úplně stejný postup hledání vázaných lokálních extrému jako v předchozí sekci. Jedinou změnou budou matice H , vektor $\mathbf{c}$ a konstanta k , konkrétně

$$H = \frac{1}{2} \sum_i H_i \quad \mathbf{c} = \frac{1}{2} \sum_i \left[(\mathbf{v}_2^{e_i} - \mathbf{v}_1^{e_i}) \times (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})^T \right] \quad k = \frac{1}{2} \sum_i \left[(\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i})^T (\mathbf{v}_2^{e_i} \times \mathbf{v}_1^{e_i}) \right]$$

Tímto máme zase šanci získat omezení do našeho systému.

V určitých případech ale ani tímto krokem nesesbíráme požadovaná tři omezení. Například když máme rovinnou síť, která má navíc rovnou ??? hranici, kontrakcí hrany na hranici bude

změna objemu sítě i obsahu hranice nulová. Máme tedy stále celou přímku možností, kam umístit výsledný vrchol. LT pro tento případ definuje poslední geometrické kritérium, ze kterého můžeme dostat nějaká omezení.

5.1.6 Optimalizace tvaru trojúhelníků

Pokud všechna předešlá kritéria selžou, pokusíme se získat omezení optimalizací tvaru jednotlivých stěn v síti. Budeme upřednostňovat rovnostranné trojúhelníky před úzkými a protáhlými – ty totiž mohou vytvářet vizuální nespojitosti, zpomalovat renderování celé sítě a zpomalit následné výpočty na zjednodušené síti (viz spektrální simplifikace??).

Tvar trojúhelníků nejlépe popíšeme délkou jejich stran. Budeme chtít, ať mají strany co nejvíc podobnou délku. Zkusme proto minimalizovat funkci

$$f_S(\mathbf{e}, \bar{\mathbf{v}}) = \sum_i \|\bar{\mathbf{v}} - \mathbf{v}_i\|^2$$

tedy součet kvadrátů délek hran mezi výsledným vrcholem $\bar{\mathbf{v}}$ a jeho sousedními vrcholy (po kontrakci hrany). Jednoduchým roznásobením se dostaneme zase do tvaru funkce (5.3):

$$f_S(\mathbf{e}, \bar{\mathbf{v}}) = \bar{\mathbf{v}}^T \sum_i I \bar{\mathbf{v}} - 2 \sum_i \mathbf{v}_i^T \bar{\mathbf{v}} + \sum_i \mathbf{v}_i^T \mathbf{v}_i \quad (5.7)$$

konkrétně s

$$H = 2 \sum_i I \quad \mathbf{c} = -2 \sum_i \mathbf{v}_i^T \quad k = 2 \sum_i \mathbf{v}_i^T \mathbf{v}_i$$

kde I značí jednotkovou matici. Stejně jako ve dvou předešlých sekcích tuto funkci minimalizujeme za podmínek $A\mathbf{x} = \mathbf{b}$ a najdeme její vázané lokální extrémy.

Pokud ani tento výpočet neurčí výslednou pozici vrcholu $\bar{\mathbf{v}}$, mohli bychom samozřejmě dál přidávat další geometrická kritéria či vybírat pozici $\bar{\mathbf{v}}$ v určitých podmnožinách $\mathbb{R}^3$, například na přímce určené hranou $\mathbf{e}$, avšak LT namísto toho vyřadí hranu z fronty úplně a nebude ji tedy možné v dalším kroku odstranit.

Nakonec se můžeme zamyslet nad tím, proč jsme vždy minimalizovali objem, obsah a délku, které byly umocněné na druhou. Druhá mocnina totiž může vnést do algoritmu nepřesnosti, když se konektivita sítě, tedy přesné rozpoložení vrcholů a stěn, změní tak, že její přesný tvar zůstane stejný. Například když nějakou stěnu jen rozdělíme na dvě části, zjevně tím nezpůsobíme změnu tvaru sítě, ale přesto součet objemů, obsahů či délek bude kvůli druhé mocnině jiný. Na druhou stranu nám druhá mocnina zjednoduší počítání odstraněním odmocniny v normě a taktéž se dokáže efektivně vypořádat s „extrémními“ hodnotami, podobně jako například metoda nejmenších čtverců. Konkrétně třeba u optimalizace tvaru trojúhelníků nám umožní srovnat délky sousedních hran vrcholu $\bar{\mathbf{v}}$, jelikož druhá mocnina bude obzvláště hodně penalizovat dlouhé hrany a vytvoří tím pádem co nejvíc rovnostranné trojúhelníky.

5.2 Cena za odstranění hrany

Pokud jsme úspěšně získali právě tři omezení pro daný vrchol $\bar{\mathbf{v}}$, čímž máme jeho přesné souřadnice, musíme nakonec definovat cenu za odstranění hrany, kterou právě počítáme. LT využívá hodnot funkcí (5.2) a (5.6), jež jsme minimalizovali a definuje velikost chyby jako lineární kombinaci těchto dvou funkcí:

$$E(\mathbf{e}, \bar{\mathbf{v}}) = \lambda f_V(\mathbf{e}, \bar{\mathbf{v}}) + (1 - \lambda) f_B(\mathbf{e}, \bar{\mathbf{v}}) L^2(\mathbf{e})$$

kde λ je libovolný reálný parametr. Funkce L značí délku hrany $\mathbf{e}$ a přidáváme ji tam proto, aby oba sčítance měly stejné jednotky, tzn. *délka na šestou*. Ve vztahu nevystupuje hodnota funkce (5.7), protože z vlastností sítě víme, že když už došlo až k optimalizaci tvaru trojúhelníků, síť bude pravděpodobně rovinná a bude tudíž mít velmi malé hodnoty funkcí $f_V(\mathbf{e}, \bar{\mathbf{v}})$ a $f_B(\mathbf{e}, \bar{\mathbf{v}})$. Tím pádem by funkce $f_S(\mathbf{e}, \bar{\mathbf{v}})$ zbytečně zvyšovala hodnotu velikosti chyby, i přestože my chceme odstraňovat hrany, které co nejméně mění objem sítě a obsah hranice. Co se týče parametru λ , LT volí hodnotu $\frac{1}{2}$, jenž jsem použil při testování algoritmu i zde.

5.3 Přepočítávání chyby po odstranění hrany

Z výše popsané LT simplifikace víme, že při zachování objemu sítě bere v úvahu sousední stěny vrcholů $\mathbf{v}_1$ a $\mathbf{v}_2$. Jelikož právě tyto stěny po odstranění hrany mění svůj tvar, je nutné přepočítat velikost chyby nejenom u hran (??? značení) sousedních s vrcholem $\bar{\mathbf{v}}$, ale i u hran sousedních se sousedními vrcholy vrcholu $\bar{\mathbf{v}}$, jelikož v jejich okolí budou změněné stěny z předchozí kontrakce hrany. (??? obrázek)

Kapitola 6

Spektrální simplifikace

Narozdíl od dvou předchozích simplifikačních algoritmů, hlavní cíl spektrální simplifikace není co nejpřesnější vizuální podoba sítě po srovnání s originálem, ale zaměřuje se především na zachování jejích spektrálních vlastností. Konkrétně se snaží zachovat vlastní čísla a vlastní vektory takzvaného diskrétního Laplaceova operátoru sítě. Čím méně změníme jeho spektrum, tím rychlejší a přesnější pak budou následné výpočty na simplifikované síti. Například mesh smoothing (neboli uhlazování sítě) nebo mesh parametrization (rozvinutí 3D sítě na 2D rovinu) pracují lépe a přesněji, čím podobnější je spektrum zjednodušené sítě. Naopak ale GH a LT mohou vyprodukovat sítě, které při následném zpracování vytvoří mnoho nepřesností. Proto postupně začaly vznikat simplifikační metody, jež se snažily zachovávat nejen „vnější“ (vizuální) vlastnosti sítě, ale i její „vnitřní“ vlastnosti. Spektrální simplifikace, vyvinuta v roce 2020, je právě jednou z nich. Předcházelo ji jen pár metod, jejichž výstupem ale nebyla zjednodušená síť, ale jen množina bodů bez stěn.

6.1 Diskrétní Laplaceův operátor

Laplaceův operátor je definován jako divergence gradientu funkce f v euklidovském prostoru, což se dá vyjádřit jako suma druhých parciálních derivací funkce dle jednotlivých proměnných

$$\Delta f = \nabla^2 f = \sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2}$$

Najdeme jej v nejrůznějších diferenciálních rovnicích popisující fyzikální jevy, například ve vlnové rovnici, rovnici proudění tepla nebo Schrödingerově rovnici. V obecnějším Riemannově prostoru existuje jeho zobecnění – Laplace–Beltramiho operátor.

My ale pracujeme s diskrétními sítěmi, proto by nám tato spojitá verze Laplaceova operátoru moc nepomohla. Podívejme se tedy na definici diskrétního Laplaceova operátoru pro 3D síť:

$$\Delta l = M^{-1}L$$

kde M je takzvaná hmotností matice a L je Laplaceova matice sítě. Obě dvě jsou reálné a velikosti $|V| \times |V|$, kde $|V|$ je počet vrcholů sítě. M je diagonální matice součtů obsahů sousedních stěn daného vrcholu. Konkrétně

$$M = \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_{|V|} \end{bmatrix} \quad A_i = \frac{1}{3} \sum_{t \in N(\mathbf{v}_i)} S(t)$$

kde $N(i)$ (označení ???) jsou sousední stěny vrcholu $\mathbf{v}_i$ a $S(t)$ je obsah stěny t . Inverzní matici z diagonální vytvoříme snadno – stačí převrátit hodnoty na diagonále, tedy M^{-1} bude obsahovat $\frac{1}{A_1}, \frac{1}{A_2}, \dots, \frac{1}{A_{|V|}}$

Laplaceova matice obecně v teorii grafů slouží jako jedna z možností reprezentace grafu v počítači. U sítí se používá její varianta s kotangensy. Každý prvek L_{ij} na i -tém řádku odpovídající $\mathbf{v}_i$ a j -tém sloupci odpovídající $\mathbf{v}_j$ má v této matici jednu z následujících tří hodnot:

$$L_{ij} = \begin{cases} w_{ij} = \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}) & \text{pokud existuje hrana } (\mathbf{v}_i, \mathbf{v}_j) \\ - \sum_{\mathbf{v}_j \in N(\mathbf{v}_i)} w_{ij} & \text{pro } i = j \\ 0 & \text{jinak} \end{cases}$$

Úhly α_{ij} a β_{ij} jsou vnitřní úhly sousední stěny dané hrany $(\mathbf{v}_i, \mathbf{v}_j)$ naproti té hraně, jak to ilustruje následující obrázek (???).

6.2 Odvození ceny za odstranění hrany

Spektrální simplifikace se snaží co nejvíce zachovat vlastní vektory diskrétního Laplaceova operátoru $\Delta l = M^{-1}L$. Označme si M, L postupně jako hmotnostní a Laplaceovu matici původní sítě a $\widetilde{M}, \widetilde{L} \in \mathbb{R}^{|\widetilde{V}| \times |\widetilde{V}|}$ jako matice simplifikované sítě. Budeme se snažit, ať platí

$$PM^{-1}Lf = \widetilde{M}^{-1}\widetilde{L}Pf$$

pro vlastní vektor $f \in \mathbb{R}^{|V|}$ operátoru Δl , kde P je takzvaná restriční matice, jež upravuje velikosti matic, aby byl zápis korektní. Přesněji musí platit $\widetilde{V} = PV$, pro matici souřadnic vrcholů původní sítě, V , a zjednodušené sítě, $\widetilde{V}$. Obecně tedy chceme, ať rozdíl mezi vlastními vektory operátoru původní a zjednodušené sítě jsou co nejmenší. Dále v textu bude popsán postup pro vytvoření matice P .

Nyní stačí uvedenou rovnici upravit pro K vlastních vektorů, které chceme co nejlépe zachovat. SP simplifikace umožňuje zadat číslo K jako vstupní parametr do programu a podle toho

určuje, kolik prvních (vzhledem k vzestupně seřazeným vlastním číslům) vlastních vektorů se bude zachovávat. Ty se totiž zaměřují na nízké frekvence a dělají síť více uhlazenou oproti vektorům s nejvyššími vlastními čísly (??? zdroj). Máme tedy matici vlastních vektorů $F \in \mathbb{R}^{|V| \times K}$ a dostáváme $PM^{-1}LF = \widetilde{M}^{-1}\widetilde{L}PF$. Ted' už jen použijeme normu, abychom z rozdílu dvou matic dostali reálné číslo, podle kterého můžeme porovnávat velikost chyby. SP simplifikace volí následující normu

$$\|X\|_{\widetilde{M}}^2 = \text{tr}(X^T \widetilde{M} X)$$

což označuje stopu matice $X^T \widetilde{M} X$, tedy součet prvků na diagonále. Velikost chyby definujeme jako normu rozdílu levé a pravé strany výše uvedené rovnice:

$$E = \|PM^{-1}LF - \widetilde{M}^{-1}\widetilde{L}PF\|_{\widetilde{M}}^2$$

Označíme si $Z = M^{-1}LF$, protože se po celou dobu simplifikace nebude měnit. Jelikož $\widetilde{M}$ je diagonální matice, můžeme normu dále upravovat:

$$\|X\|_{\widetilde{M}}^2 = \text{tr}(X^T \widetilde{M} X) = \text{diag}(\widetilde{M})^T \text{diag}(X X^T) = \sum_i \widetilde{M}_i \|X_i\|^2$$

kde $\widetilde{M}_i$ značí i -tý diagonální prvek matice $\widetilde{M}$ a X_i je i -tý řádek matice X (tzn. i značí i -tý vrchol sítě). Dosazením dostaneme

$$E = \sum_i \widetilde{M}_i \|(PZ - \widetilde{M}^{-1}\widetilde{L}PF)_i\|^2 = \sum_i E_i \quad (6.1)$$

Máme tedy velikost chyby pro každý vrchol sítě. Pokud nyní budeme chtít odstranit hranu, můžeme si všimnout, že po jejím odstranění se změní hmotnostní a Laplaceova matice sítě pouze pro sousední (značení ???) vrcholy výsledného vrcholu $\bar{\mathbf{v}}$, jelikož pouze mezi nimi dojde ke změně geometrie jednotlivých stěn. To znamená, že jediné těmto vrcholům se změní velikost chyby. Cenu pro odstranění jedné hrany $\mathbf{e} = (\mathbf{v}_1, \mathbf{v}_2)$ definujeme jako rozdíl celkové chyby všech vrcholů v síti **po** možné kontrakci hrany a **před** ní. Z našeho pozorování pak víme, že stačí počítat jen okolí hrany $\mathbf{e}$, čímž výpočet dost zjednodušíme, jak ukazuje následující rovnost.

$$\begin{aligned} E(\mathbf{e}, \bar{\mathbf{v}}) &= E^{po} - E^{před} \\ &= \left(\sum_{i \in N(\bar{\mathbf{v}})} E_i^{po} + \sum_{i \notin N(\bar{\mathbf{v}})} E_i^{po} \right) - \left(\sum_{i \in N(\mathbf{v}_1, \mathbf{v}_2)} E_i^{před} + \sum_{i \in N(\mathbf{v}_1, \mathbf{v}_2)} E_i^{před} \right) \\ &= \sum_{i \in N(\bar{\mathbf{v}})} E_i^{po} - \sum_{i \in N(\mathbf{v}_1, \mathbf{v}_2)} E_i^{před} \end{aligned}$$

Musíme si tudíž uchovávat velikost chyby z minulých kroků pro každý vrchol, který se aktuálně na síti nachází. Ted' už nám jen stačí najít ideální pozici pro vrchol $\bar{\mathbf{v}}$.

6.3 Hledání ideální pozice vrcholu

Stejně jako u předešlých algoritmů máme více možností jak hledat ideální pozici. Nejlepší by samozřejmě bylo se zaměřit na celý 3D prostor, to je ale pro tento simplifikační algoritmus relativně výpočetně náročné. Ač méně přesné, hledání ideální pozice pouze na hraně $\mathbf{e}$ je o něco rychlejší.

Jelikož funkce erroru $E(\mathbf{e})$ je v celém 3D prostoru zase kvadrikou, na hraně $\mathbf{e}$ (obecně na přímce určené touto hranou) se bude jednat o parabolu a tu musíme minimalizovat, tedy najít její minimum. Nejjednodušeji to uděláme výpočtem velikosti chyby u 3 bodů na naší hraně, podle kterých pak už jednoduše sestavíme rovnici paraboly a derivací najdeme minimum. Přímku určenou hranou $\mathbf{e} = (\mathbf{v}_1, \mathbf{v}_2)$ si vyjádříme v jejím parametrickém tvaru podle parametru $\alpha \in \mathbb{R}$:

$$p(\alpha) = \mathbf{v}_1 + (\mathbf{v}_2 - \mathbf{v}_1)\alpha = \mathbf{v}_1(1 - \alpha) + \mathbf{v}_2(\alpha) \quad (6.2)$$

Parametrem α si tudíž můžeme označit i hodnotu chyby v daném bodě na přímce, což nám vytvoří onu parabolu:

$$E(\mathbf{e}, p(\alpha)) \stackrel{\text{ozn.}}{=} E(\mathbf{e}, \alpha) = a(\alpha^2) + b(\alpha) + c$$

Nyní si vybereme 3 body. Pro jednoduchost zvolíme $\mathbf{v}_1$, $\mathbf{v}_2$ a bod uprostřed hrany $\mathbf{e}$, to znamená postupně body $p(0)$, $p(1)$ a $p(0.5)$. Vypočteme-li velikost chyby v těchto bodech, získáme soustavu 3 rovnic

$$\begin{aligned} E(\mathbf{e}, 0) &= c \\ E(\mathbf{e}, 0.5) &= 0.25a + 0.5b + c \\ E(\mathbf{e}, 1) &= a + b + c \end{aligned}$$

ze které už snadno zjistíme koeficienty a , b a c a stacionární bod této paraboly ležící v $\alpha^* = \frac{-b}{2a}$. Vypočteme proto chybu $E(\mathbf{e}, \alpha^*)$, všechny hodnoty chyby mezi sebou porovnáme a vybereme tu nejnižší, čímž také ověříme, zdali je ve stacionárním bodu skutečně minimum. Do proměnné α^* zapíšeme pozici, ve které nastane minimální chyba. Máme tudíž výsledný vrchol $\bar{\mathbf{v}}$ v bodě $p(\alpha^*)$ i s celkovou cenou za odstranění hrany $\mathbf{e}$.

6.4 Restrikční matice

Nyní už zbývá jen odvození restrikční matice P . Tu si můžeme vyjádřit jako součin jednotlivých restrikčních matic v každém kroku simplifikace

$$P = Q_n Q_{n-1} \dots Q_2 Q_1$$

pro n odstraněných hran. Podobně jako u celé matice P budeme i od každého Q_k vyžadovat, ať platí $\tilde{V}_k = Q_k \tilde{V}_{k-1}$ kde $\tilde{V}_k$ bude matice vrcholů po kontrakci k hran, tzn. po odstranění k vrcholů (pozn: pak $\tilde{V}_0 = V$). Konkrétně při odstranění hrany $\mathbf{e} = (\mathbf{v}_i, \mathbf{v}_j)$ se vytvořil nový vrchol $\bar{\mathbf{v}}$, jež je nějakou lineární kombinací dvou původních vrcholů (z rovnice (6.2)):

$$\bar{\mathbf{v}} = \mathbf{v}_i(1 - \alpha) + \mathbf{v}_j(\alpha)$$

Pro jednoduchost implementace předpokládejme, že vrchol $\mathbf{v}_i$ se při kontrakci hrany přesune na vrchol $\bar{\mathbf{v}}$, to znamená, že v matici $\tilde{V}_k$ zůstane. Naopak $\mathbf{v}_j$ při odstraňování hrany $\mathbf{e}$ zahodíme, proto jej budeme chtít z matice odstranit. Po jednotlivých restričních maticích Q_k proto požadujeme, ať sečte násobky i -tého a j -tého řádku matice $\tilde{V}_{k-1}$ (tzn. souřadnice vrcholu $\mathbf{v}_i$ a $\mathbf{v}_j$) do i -tého řádku matice $\tilde{V}_k$ (což budou souřadnice vrcholu $\bar{\mathbf{v}}$):

$$\tilde{V}_k = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_{i-1} \\ (1 - \alpha)\mathbf{v}_i + \alpha \mathbf{v}_j \\ \mathbf{v}_{i+1} \\ \vdots \\ \mathbf{v}_{j-1} \\ \mathbf{v}_{j+1} \\ \vdots \\ \mathbf{v}_N \end{bmatrix} = \begin{bmatrix} & & & i & & j & & \\ 1 & 0 & \dots & 0 & \dots & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & 0 & \dots & 1 - \alpha & \dots & \alpha & \dots & 0 \\ \vdots & \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} v_{1,x} & v_{1,y} & v_{1,z} \\ \vdots & \vdots & \vdots \\ v_{i-1,x} & v_{i-1,y} & v_{i-1,z} \\ v_{i,x} & v_{i,y} & v_{i,z} \\ v_{i+1,x} & v_{i+1,y} & v_{i+1,z} \\ \vdots & \vdots & \vdots \\ v_{j-1,x} & v_{j-1,y} & v_{j-1,z} \\ v_{j,x} & v_{j,y} & v_{j,z} \\ v_{j+1,x} & v_{j+1,y} & v_{j+1,z} \\ \vdots & \vdots & \vdots \\ v_{N,x} & v_{N,y} & v_{N,z} \end{bmatrix} = Q_k \tilde{V}_{k-1}$$

V praxi vždy budeme počítat velikost chyby při potenciálním odstranění jedné hrany, tudíž místo matice P ve výpočtu použijeme obecně jen matici Q pro danou kontrakci hrany. Z rovnice (6.1) získáme vztah, který použijeme v implementaci:

$$E_m^{po} = \tilde{M}_m \|(QZ - \tilde{M}^{-1} \tilde{L} Q F)_m\|^2$$

Matice $\tilde{M}$ a $\tilde{L}$ závisí na parametru α , tedy na přesné změně geometrie v síti posunutím $\mathbf{v}_i \rightarrow \bar{\mathbf{v}}$. Je nutné je spočítat pro $N(\mathbf{v}_i, \mathbf{v}_j)$ (značení???) kromě samotného vrcholu $\mathbf{v}_j$, ten se bude odstraňovat a v celkové chybě tak nehraje žádnou roli. Navíc, jelikož v euklidovské normě bereme pouze řádek odpovídajícího vrcholu, na matici Q dojde pouze když $m = i$, tedy když zrovna počítáme chybu u „posunutého“ $\mathbf{v}_i$. V tomto případě stačí udělat lineární kombinaci příslušných řádků matic Z a F ,

to znamená

$$\begin{aligned}(QZ)_i &= (1 - \alpha)Z_i + \alpha Z_j \\ (QF)_i &= (1 - \alpha)F_i + \alpha F_j\end{aligned}$$

6.5 Přepočítání hran a úprava matic

Po každém odstranění hrany, musíme podobně jako u GH simplifikace upravit určité matice. Pro nás jsou teď důležité F a Z , jelikož ty spoléhají na restriční matici Q , aby měly správnou velikost a mohli jsme je násobit s upravenou hmotnostní a Laplaceovou maticí, které se budou v každé iteraci zmenšovat. Stejně jako u výpočtu velikosti chyby proto musíme sečíst dané řádky reprezentující kontrakci hrany e a výsledek vložit do matice:

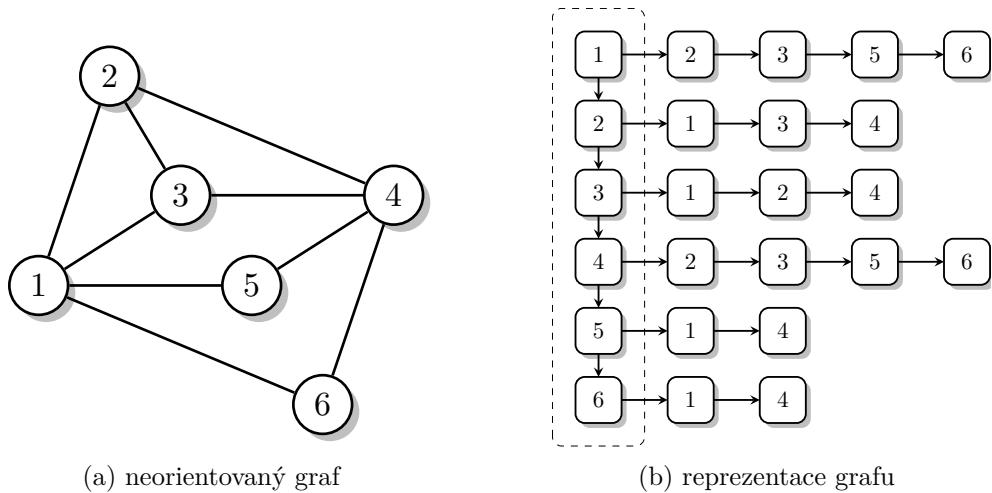
$$\begin{aligned}Z_i &\leftarrow (1 - \alpha^*)Z_i + \alpha^* Z_j \\ F_i &\leftarrow (1 - \alpha^*)F_i + \alpha^* F_j\end{aligned}$$

Parametr α^* , kterým jsme vyjádřili pozici vrcholu $\bar{v}$ na přímce p dané hranou e , kde je velikost chyby minimální, bude nutné pro každou hranu uchovávat.

Co se týče přepočítávání hran, spektrální simplifikace je v tomto ohledu ještě výpočetně náročnější než GH nebo LT. Když dojde ke kontrakci hrany, změní se geometrie sítě pro mezi vrcholy $N_1(\bar{v})$ (značení, obrázek ???). To způsobí, že hmotnostní i Laplaceova matice pro sousedy vrcholu $\bar{v}$ budou jiné. Z toho důvodu dojde ke změně velikosti chyby i u hran vzdálených ob 2 vrcholy od $\bar{v}$. Musíme tudíž přepočítat sousední hrany $N_2(\bar{v})$

Kapitola 7

Pravidelnými ovce dosavadní



Obrázek 7.1: Ukázkový obrázek se dvěma podobrázky

Tabulka 7.1: Experimentální výsledky

Pokus #	α	Algoritmus 1		Algoritmus 2	
		β	γ	δ	χ
1	20,714	50,0798	-91	-10	70,905
2	71,8653	-54,2	-48,7	11,536	33,551
3	50,33319	-53,63	-10	-14,9	-98
4	-68,98	87,2712	-89,74	-30	-9,47
5	7,934	77,214	55,457	-57,5	-13,2
6	-14,68	59,108	23,62571	-10	68,548
7	18,498	80,002	4,888	44,909	-50
8	3,746	25,59786	99,8605	-80,8	23,9323
9	46,7614	85,043	-95	8,5701	49,5099
10	-58,8	-38,8	87,8912	98,18994	-94,4

Kapitola 8

Technické detaily

8.1 Křížové odkazy

Odborné texty, mezi které lze počítat i bakalářské, diplomové a disertační práce, obvykle obsahují množství křížových odkazů odkazujících na nejrůznější části textu:

kapitoly – například odkaz na kapitolu ???. Pokud odkazujeme na kapitolu, která je značně vzdálená od současné stránky, bývá dobrým zvykem k odkazu na číslo kapitoly přidat ještě i odpovídající číslo stránky, jako například pokud odkazujeme na kapitolu 1 na straně 7.

obrázky – například odkaz na obrázky ??, ?? a ??. Menší, vzájemně související obrázky můžeme sdružit do jednoho obrázku a odkazovat se buď na menší obrázky, například 7.1a a 7.1b, nebo na celkový obrázek, spíše řekneme, ilustraci 7.1.

tabulky – například odkaz na tabulky 7.1 a ??. Podobně jako u obrázků můžeme menší tabulky ?? a ?? sdružit do jedné společné a odkazovat se na obě menší tabulky jednotně, jako například na tabulku ??.

rovnice – odkazy na rovnice se obvykle uzavírají do kulatých závorek, jako například v odkazech na rovnice (??), (??) nebo (??).

výpisy zdrojového kódu – například odkaz na výpis ??. Výpis ?? je ukázkou výpisu v jiném programovacím jazyce, v tomto případě v jazyce Python, než je výchozí jazyk C++. Samozřejmě se lze odkazovat i na velmi dlouhé výpisy, jako například výpis ?? na straně ?? v příloze ??, který je načítán z externího souboru.

8.2 Jak citovat

Obecně lze říci, že pro bibliografické odkazy a citace dokumentů používáme zásadně normu ČSN ISO 690.

8.2.1 Odkaz v textu

Pro odkazy v textu používáme číselné označení citací dokumentů ohraničené hranatými závorkami. Takže například můžeme citovat časopisecké *články* [2, 3, 4, 5, 6, 7], *knihy* [8, 9, 10, 11, 12, 13, 14, 15, 16], *periodika* [17], *bakalářské, diplomové či diserteční práce* [18], *patenty* [19, 20, 21, 22], *online zdroje* [23, 24, 25, 26, 27] či *manuály* [28].

8.2.2 Seznam citací

Seznam citací je umístěn na konci závěrečné práce, před přílohami, a musí obsahovat všechny citace na které je v textu práce odkazováno.

8.3 Překlad

Pro kompilaci této ukázkové práce úplně od počátku¹ je nutné provést několik spuštění pdfL^AT_EXu a programu Biber v následujícím pořadí:

```
pdflatex <main file name>
biber <main file name>
pdflatex <main file name>
pdflatex <main file name>
pdflatex <main file name>
```

¹Anglicky build from scratch

Kapitola 9

Závěr

Tohle je závěr.

Literatura

1. *I love you Coffee* [online]. [cit. 2019-04-24]. Dostupné z: <http://preetiyacoffee.blogspot.com/>.
2. HERRMANN, Wolfgang A.; ÖFELE, Karl; SCHNEIDER, Sabine K.; HERDTWECK, Eberhardt; HOFFMANN, Stephan D. A carbocyclic carbene as an efficient catalyst ligand for C–C coupling reactions. *Angew. Chem. Int. Ed.* 2006, roč. 45, č. 23, s. 3859–3862.
3. BERTRAM, Aaron; WENTWORTH, Richard. Gromov invariants for holomorphic maps on Riemann surfaces. *J. Amer. Math. Soc.* 1996, vol. 9, no. 2, s. 529–571.
4. MOORE, Gordon E. Cramming more components onto integrated circuits. *Electronics*. 1965, vol. 38, no. 8, s. 114–117.
5. YOON, Myeong S.; RYU, Dowook; KIM, Jeongryul; AHN, Kyo Han. Palladium pincer complexes with reduced bond angle strain: efficient catalysts for the Heck reaction. *Organometallics*. 2006, roč. 25, č. 10, s. 2409–2411.
6. SIGFRIDSSON, Emma; RYDE, Ulf. Comparison of methods for deriving atomic charges from the electrostatic potential and moments. *Journal of Computational Chemistry*. 1998, vol. 19, no. 4, s. 377–395. Dostupné z DOI: 10.1002/(SICI)1096-987X(199803)19:4<377::AID-JCC1>3.0.CO;2-P.
7. BAEZ, John C.; LAUDA, Aaron D. Higher-Dimensional Algebra V: 2-Groups. *Theory and Applications of Categories*. 2004, vol. 12, s. 423–491. Dostupné z arXiv: [math/0307200v3](https://arxiv.org/abs/math/0307200v3).
8. WILDE, Oscar. *The Importance of Being Earnest: A Trivial Comedy for Serious People*. Leonard Smithers and Company, 1899. English and American drama of the Nineteenth Century. Dostupné z Google Books: [4HIWAAAAAYAAJ](https://books.google.com/books?id=4HIWAAAAAYAAJ).
9. NIETZSCHE, Friedrich. *Sämtliche Werke: Kritische Studienausgabe*. Bd. 1, Die Geburt der Tragödie. Unzeitgemäße Betrachtungen I–IV. Nachgelassene Schriften 1870–1973. 2. Aufl. Hrsg. von COLLI, Giorgio; MONTINARI, Mazzino. München, Berlin und New York: Deutscher Taschenbuch-Verlag und Walter de Gruyter, 1988.

10. AVERROES. *The Epistle on the Possibility of Conjunction with the Active Intellect by Ibn Rushd with the Commentary of Moses Narboni*. Ed. by BLAND, Kalman P. Trans. by BLAND, Kalman P. New York: Jewish Theological Seminary of America, 1982. Moreshet: Studies in Jewish History, Literature and Thought, no. 7.
11. HAMMOND, Christopher. *The basics of crystallography and diffraction*. Oxford: International Union of Crystallography and Oxford University Press, 1997.
12. COTTON, Frank Albert; WILKINSON, Geoffrey; MURILLIO, Carlos A.; BOCHMANN, Manfred. *Advanced inorganic chemistry*. 6th ed. Chichester: Wiley, 1999.
13. KNUTH, Donald E. *Computers & Typesetting*. Vol. A, The T_EXbook. Reading, Mass.: Addison-Wesley, 1984.
14. GERHARDT, Michael J. *The Federal Appointments Process: A Constitutional and Historical Analysis*. Durham and London: Duke University Press, 2000.
15. GONZALEZ, Ray. *The Ghost of John Wayne and Other Stories*. Tucson: The University of Arizona Press, 2001. ISBN 0-816-52066-6.
16. GOOSSENS, Michel; MITTELBACH, Frank; SAMARIN, Alexander. *The LaTeX Companion*. 1st ed. Reading, Mass.: Addison-Wesley, 1994.
17. *Computers and Graphics*. Semantic 3D Media and Content. 2011, roč. 35, č. 4. ISSN 0097-8493.
18. GEER, Ingrid de. *Earl, Saint, Bishop, Skald – and Music: The Orkney Earldom of the Twelfth Century. A Musicological Study*. Uppsala, 1985. Dis. pr. Uppsala Universitet.
19. KOWALIK, F.; ISARD, M. *Estimateur d'un défaut de fonctionnement d'un modulateur en quadrature et étage de modulation l'utilisant*. Vynálezce: F. KOWALIK; M. ISARD. **publication**: 1995-01-11. Francouzská patentová žádost 9500261.
20. ALMENDRO, José L.; MARTÍN, Jacinto; SÁNCHEZ, Alberto; NOZAL, Fernando. *Elektromagnetisches Signalhorn*. Vynálezce: José L. ALMENDRO; Jacinto MARTÍN; Alberto SÁNCHEZ; Fernando NOZAL. Publ.: 1998. FR, GB, DE. EU-29702195U.
21. HUGHES AIRCRAFT COMPANY. *High-Speed Digital-to-RF Converter*. Vynálezce: Ronald E. SORACE; Victor S. REINHARDT; Steven A. VAUGHN. Publ.: 1997-09-16. US patent 5668842.
22. ROBERT BOSCH GMBH; DAIMLER CHRYSLER AG; BAYERISCHE MOTOREN WERKE AG. *Elektrische Einrichtung und Betriebsverfahren*. Vynálezce: Xaver LAUFENBERG; Dominique EYNIUS; Helmut SUELZLE; Stephan USBECK; Matthias SPAETH; Miriam NEUSER-HOFFMANN; Christian MYRZIK; Manfred SCHMID; Franz NIETFIELD; Alexander THIEL; Harald BRAUN; Norbert EBNER. Publ.: 2006-09-13. Evropský patent 1700367.
23. CTAN: *The Comprehensive TeX Archive Network* [online]. 2006. [cit. 2006-10-01]. Dostupné z: <http://www.ctan.org>.

24. WASSENBERG, Jan; SANDERS, Peter. *Faster Radix Sort via Virtual Memory and Write-Combining*. 2010-08-17. Version 1. Dostupné z arXiv: 1008.2849v1 [cs.DS].
25. ITZHAKI, Nissan. *Some remarks on 't Hooft's S-matrix for black holes*. 1996-03-11. Version 1. Dostupné z arXiv: hep-th/9603067.
26. MARKEY, Nicolas. *Tame the BeaST: The B to X of BibTeX* [online]. 2005-10-16. Version 1.3 [cit. 2006-10-01]. Dostupné z: http://mirror.ctan.org/info/bibtex/tamethebeast/ttb_en.pdf.
27. BAEZ, John C.; LAUDA, Aaron D. *Higher-Dimensional Algebra V: 2-Groups*. 2004-10-27. Version 3. Dostupné z arXiv: math/0307200v3.
28. *The Chicago Manual of Style: The Essential Guide for Writers, Editors, and Publishers*. 15th ed. Chicago, Ill.: University of Chicago Press, 2003. ISBN 0-226-10403-6.